“시간이 아깝지 않은”

SK네트웍스 Family AI캠프 교재1

강사 이영배

강사 이력

1. 2018년 ~ 2022년 10월 : 데이터분석, 머신 러닝, 딥러닝 개발자

2. 2021년 2월 대학원 졸업

3. 개발 경력
   ① Na 배설량 분석 및 배설량 회귀 방정식 도출(분당 서울대병원)
   ② 교육부 학생 건강검사 표본통계 분석을 통한 어린이 비만 지수 예측 인공지능 모델 개발(중소벤처기업부 과제)
   ③ 휴대폰 전파 세기와 기상청 기상 정보 분석을 통한 강우량 예측 인공지능 모델 개발(중소벤처기업부 과제)
   ④ 전통식품 발효 과정 예측 인공 지능 모델 개발(2022년도 NIA 데이터 구축 사업)
   ⑤ 신속 항원 진단 키트 검사 결과 분석 AI 모델 개발(자사 솔루션)
   ⑥ 한글 특허, 영문 특허의 특허 요약 언어 모델 개발(과학기술정보통신부 연구산업육성사업)
   ⑦ 한글 문서 요약 언어 모델 개발(자사 솔루션, 공공 기관 납품용)
   ⑧ 딥러닝 기반 의류 가상 피팅 애플리케이션 개발(2020년 중소벤처기업부 예비창업패키지 인공지능분야 선정)

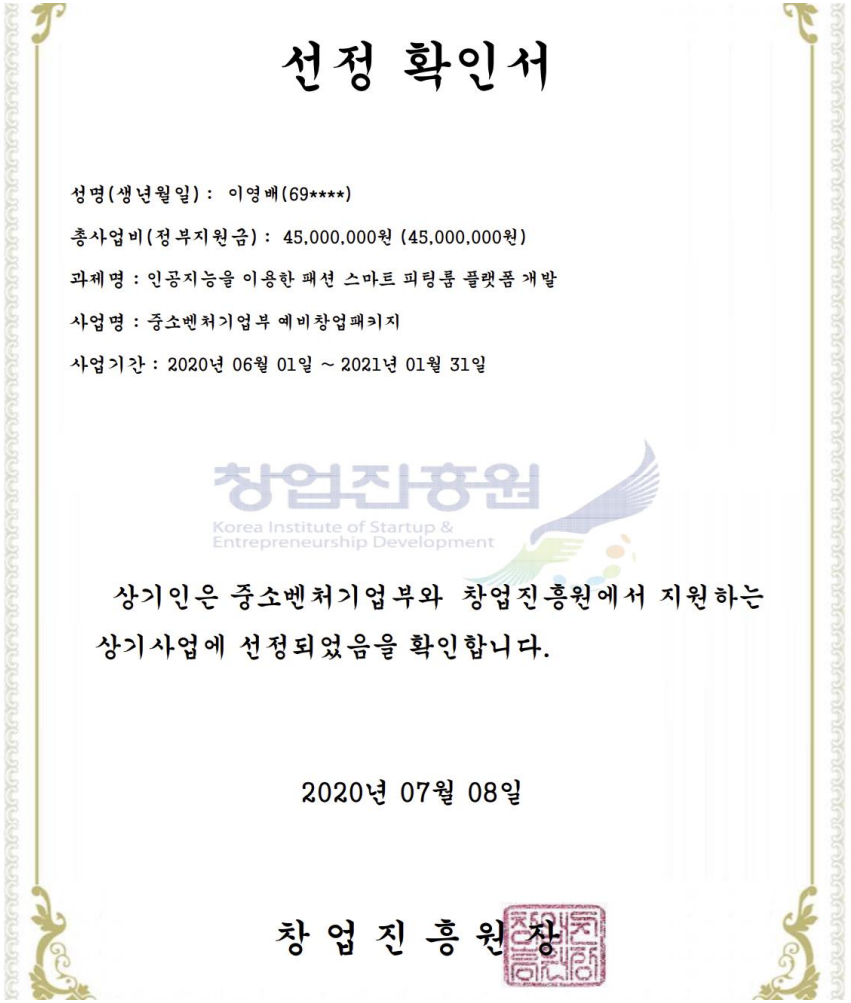
4. KCI 논문 등재(2021년 10월)
   : 음표 임베딩과 마디 임베딩을 이용한 곡의 생성 및 정량적 평가 방법(정보처리학회논문지:소프트웨어 및 데이터공학)

5. 중소벤처기업부 예비창업패키지 인공지능부분 선정(2020년, 인공지능을 이용한 패션 스마트 피팅룸 플랫폼 개발)

6. 강의 경력
   ① 2023년 ~ 2025년 : 삼성멀티캠퍼스 KDT 2023, 2024, 2025, 2026년도 주강사,
   아시아경제교육센터 KDT 2024, 2025년도 주강사
   ② 2022년 11월 ~ 현재 : 메가스터디 IT 아카데미 데이터 분석 / 머신 러닝 / 딥러닝 / AI Agent 과정 진행

강사 이력

7. 2020년도 중소벤처기업부 예비창업패키지 선정(정부지원금: 4천5백만원)



선정 확인서

성명(생년월일) : 이영배(69****)

총사업비(정부지원금) : 45,000,000원 (45,000,000원)

과제명 : 인공지능을 이용한 패션 스마트 피팅룸 플랫폼 개발

사업명 : 중소벤처기업부 예비창업패키지

사업기간 : 2020년 06월 01일 ~ 2021년 01월 31일

창업진흥원
Korea Institute of Startup & Entrepreneurship Development

상기인은 중소벤처기업부와 창업진흥원에서 지원하는 상기사업에 선정되었음을 확인합니다.

2020년 07월 08일

창 업 진 흥 원 장

강사 이력

8. 2021년 KCI 딥러닝 논문 등재

한국정보처리학회
Korea Information Processing Society

(우)04376 서울시 용산구 한강대로109, 1002호(한강로2가 용성비즈텔) / 전화(02)2077-1414 / 팩스(02)2077-1472

게 재 증 명 서

2026년 1월 22일

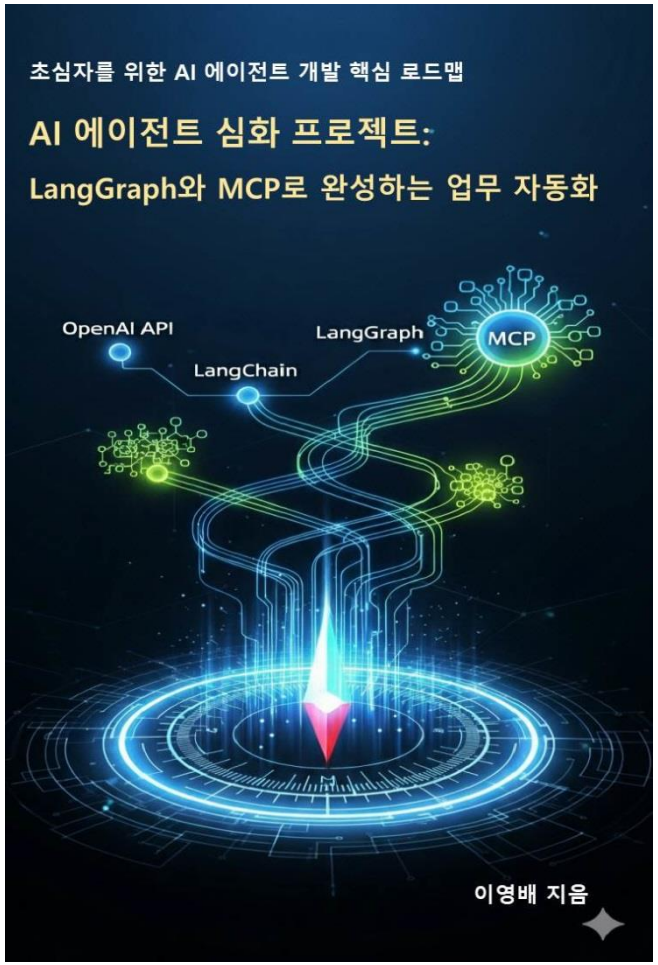
게재증명 제 2026 - 001호
논문제목 : 음표 임베딩과 마디 임베딩을 이용한곡의 생성 및 정량적 평가 방법
저 자 : 이영배, 정성훈
게재지명 : KIPS Transactions on Software and Data Engineering
게재호수 : 제 10권 제 11호(2021년 11월)
게재면수 : pp.483 ~ 490 (8페이지)
발 행 일 : 2021년 10월 31일

상기의 논문은 사단법인 한국정보처리학회의 논문 심사위원회에서 심사되어 「KIPS Transactions on Software and Data Engineering」 게재된 논문임을 증명합니다. 끝.

강사 이력

9. 2025년 9월, 10월: AI Agent 도서 2권 출간





강사 이력

10. 2026년 대한민국 AI 대표 기업 중 `업스테이지(Upstage)` AI 인증 강사 취득



수업 진행 방식

필수 이론에 대한 자세한 설명 + 단원 별 충분한 실습

- 멀티 에이전트 적용 시 핵심 변화 포인트 4가지

1) 노드의 세분화: 1개의 독점 노드 → 3개의 전문 노드(에이전트)로 분할

① 기존 rag_node 하나가 하던 다양한 일들을 정확하게 3개의 async def 독립 노드로 쪼개어 분업화 달성

② rephrase_node (질문 재구성): 대화 기록을 파악해 "그거 전액 다 받으려면 조건이 뭐야?"를 "수강 신청한 강의를 전액 환불 받으려면 조건이 뭐야?"로 명확하게 바꾼 뒤 '상태'의 'rephrased_quer

③ retriever_node (문서 검색): '상태'에 담긴 rephrased_query를 꺼내어 벡터저장소

매뉴얼 본문을 '상태'의 'context' 필드에 담는다.

④ answer_node (답변 생성): 'context' 필드에 담긴 매뉴얼 본문과 원본 질문을 조

정중한 답변 메시지를 생성한다.

LangGraph 에이전트: 멀티 에이전트 적용 실습

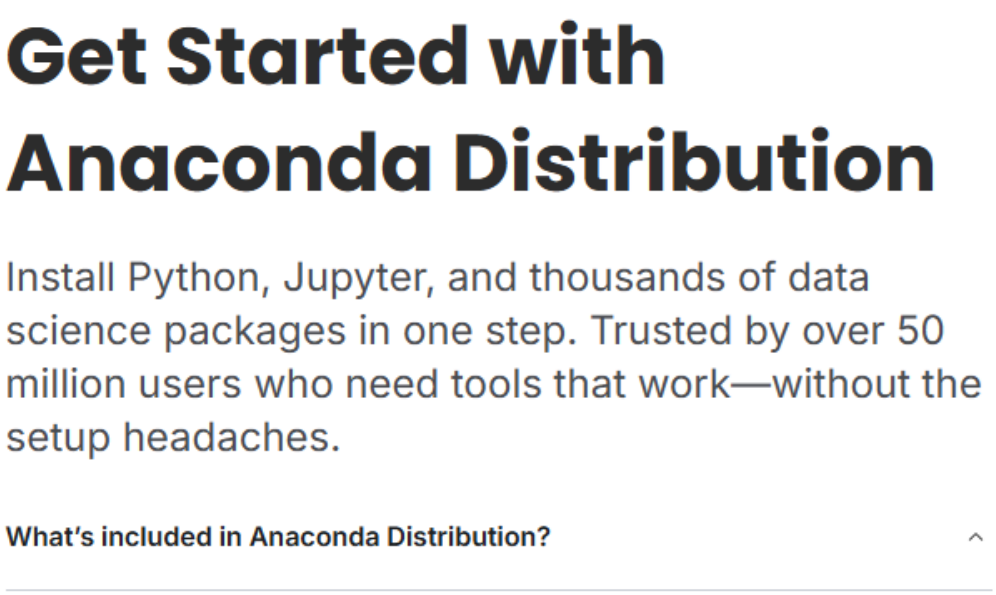
- 고객 응대 Chatbot Agent 업그레이드(3)

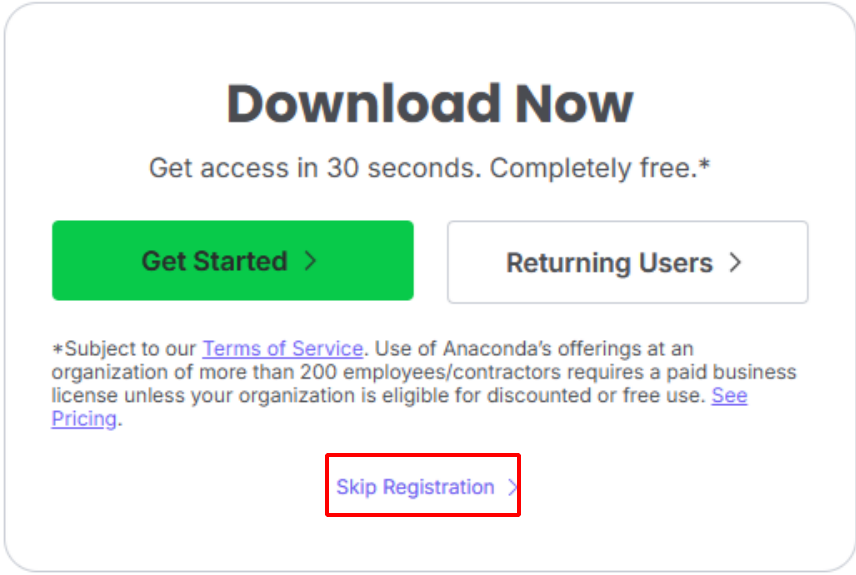


01. 실습 환경

미니콘다(miniconda) 설치

- 미니콘다 : 파이썬 패키지 및 가상 환경을 관리하는 효율적인 도구
- 미니콘다 다운로드(https://www.anaconda.com/download)





실습 환경 구축하기 – win11에 miniconda 설치

1. 공식 페이지에서 다운로드 합니다. 다운로드한 .exe 파일을 더블 클릭하여 설치를 시작하고, Next를 클릭합니다.
2. 라이선스 동의 화면에서 I Agree를 클릭합니다.
3. 설치 유형 선택 (Installation Type): Just Me (recommended)를 선택하세요.
4. 설치 경로 선택 (Choose Install Location)
   - 기본으로 제안된 경로를 그대로 사용하는 것을 권장합니다. Next를 클릭합니다.
5. 고급 설치 옵션 (Advanced Installation Options) - [중요]
   1) Add Miniconda3 to my PATH environment variable **(체크 X)**
   2) Register Miniconda3 as my default Python: 이 옵션은 체크된 상태를 유지하는 것이 좋습니다.
   3) 설정이 완료되면 Install 버튼을 클릭하여 설치를 시작합니다.
6. 설치 완료
   - 설치가 완료되면 Next를 누르고, 마지막 화면에서 Finish를 클릭하여 설치 마법사를 종료합니다.

가상 환경 설치

1. 윈도우 시작(또는 찾기) → anaconda 입력 → Anaconda Prompt 실행

2. 가상 환경 설치 : conda create –n agent python=3.12

```
(base) C:\Users\YB>conda create -n agent python=3.12
Channels:
 - defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: C:\Users\YB\anaconda3\envs\agent

  added / updated specs:
    - python=3.12


The following NEW packages will be INSTALLED:

  bzip2              pkgs/main/win-64::bzip2-1.0.8-h2bbff1b_6
  ca-certificates    pkgs/main/win-64::ca-certificates-2025.2.25-haa95532_0
  expat              pkgs/main/win-64::expat-2.7.1-h8ddb27b_0
  libffi             pkgs/main/win-64::libffi-3.4.4-hd77b12b_1
  openssl            pkgs/main/win-64::openssl-3.0.16-h3f729d1_0
  pip                pkgs/main/noarch::pip-25.1-pyhc872135_2
  python             pkgs/main/win-64::python-3.12.11-h716150d_0
  setuptools         pkgs/main/win-64::setuptools-78.1.1-py312haa95532_0
  sqlite             pkgs/main/win-64::sqlite-3.45.3-h2bbff1b_0
  tk                 pkgs/main/win-64::tk-8.6.14-h5e9d12e_1
  tzdata             pkgs/main/noarch::tzdata-2025b-h04d1e81_0
  vc                 pkgs/main/win-64::vc-14.42-haa95532_5
  vs2015_runtime     pkgs/main/win-64::vs2015_runtime-14.42.34433-hbfb602d_5
  wheel              pkgs/main/win-64::wheel-0.45.1-py312haa95532_0
  xz                 pkgs/main/win-64::xz-5.6.4-h4754444_1
  zlib               pkgs/main/win-64::zlib-1.2.13-h8cc25b3_1


Proceed ([y]/n)?
```

가상 환경 설치

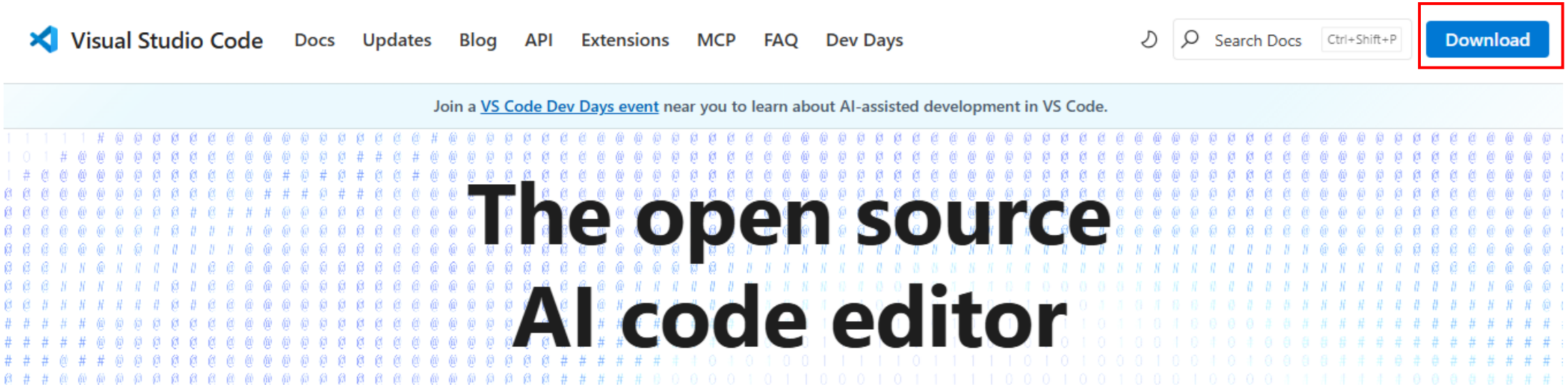
3. 가상 환경 활성화 : conda activate agent

4. 필요한 라이브러리 설치: pip install pandas matplotlib plotly streamlit jupyter ipykernel

5. miniconda3 설치 폴더 → 새 폴더 생성 : work

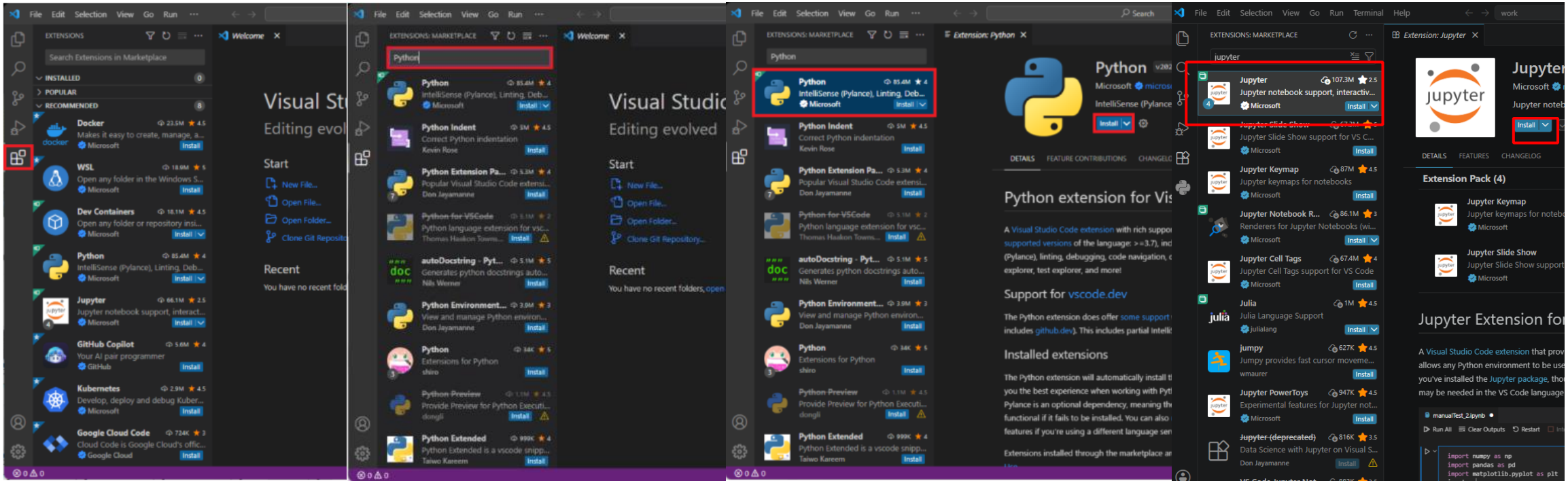
실습 환경 구축하기 – Visual Studio Code 설치

- Visual Studio Code (VS Code) : Microsoft에서 개발한 무료 코드 편집기
- VS Code 다운로드(https://code.visualstudio.com/)



실습 환경 구축하기 – 가상 환경 연동

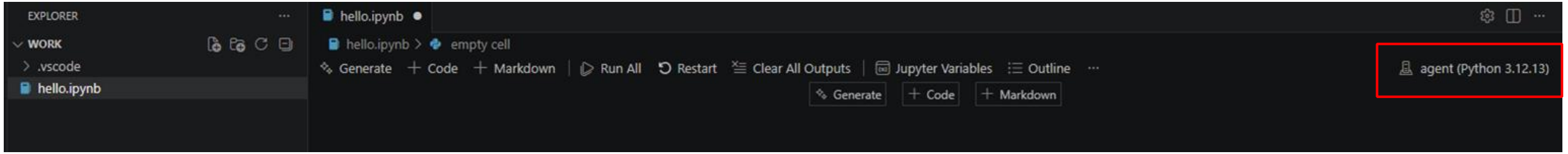
1. VScode의 좌측 툴바에서 'Extensions' 선택
2. 상단의 'Search Extensions in Marketplace' 창에 'Python', 'Jupyter ' 입력
3. 검색 결과에서 'Python' 선택 후 우측 창에서 'Install' 버튼 클릭, 'Jupyter ' 선택 후 'Install' 버튼 클릭



실습 환경 구축하기 – 가상 환경 연동

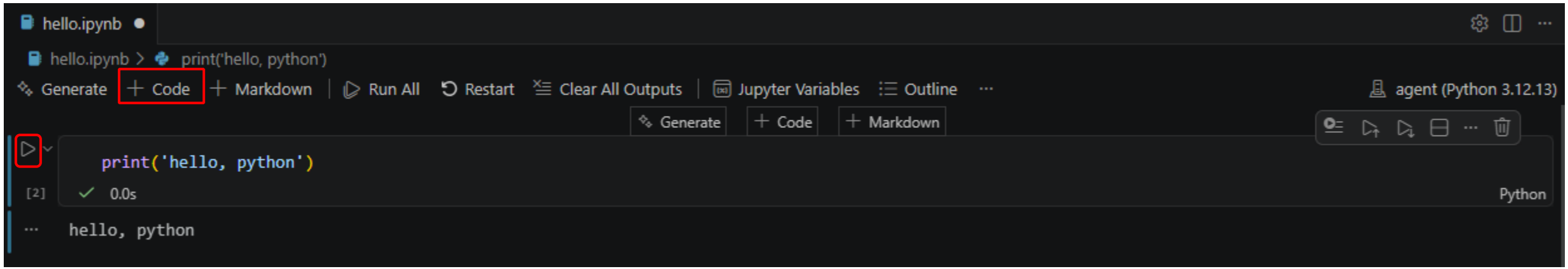
4. VS Code를 실행하고 VS Code 메뉴에서 'File' > 'Open Folder'를 선택하여 앞서 생성한 work 폴더를 엽니다.
5. 좌측 메뉴 탐색기(Explorer)에서 work 폴더 오른쪽에 있는 'New File'을 클릭하여 hello.ipynb 파일을 생성합니다.
6. 노트북 파일 우측 상단의 [커널 선택 (Select Kernel)] 버튼을 클릭하고, [Python Environments...]를 선택합니다.
7. 가상 환경 "agent (Python 3.12)"를 클릭합니다. 이렇게 하면 VS Code는 해당 프로젝트의 모든 작업을 agent 가상 환경에서 실행하고, 설치된 라이브러리를 인식하게 됩니다.

.



실습 환경 구축하기 – 가상 환경 연동

8. 노트북 파일 상단에 있는 "+ Code" 버튼을 눌러 코드 입력 창을 생성하고 "hello, python " 을 입력한 후 입력 창 왼쪽에 있는 실행 버튼을 누르거나 아니면 Shift + Enter를 눌러 코드를 실행하고, 결과가 출력되는지 확인하여 개발 환경이 올바르게 설정되었음을 검증합니다.



실습 환경 구축하기 – MySQL 설치

- MySQL : 오픈 소스 관계형 데이터베이스 관리 시스템(RDBMS)
- MySQL 다운로드(https://downloads.mysql.com/archives/community/)

MySQL Product Archives

MySQL Community Server (Archived Versions)

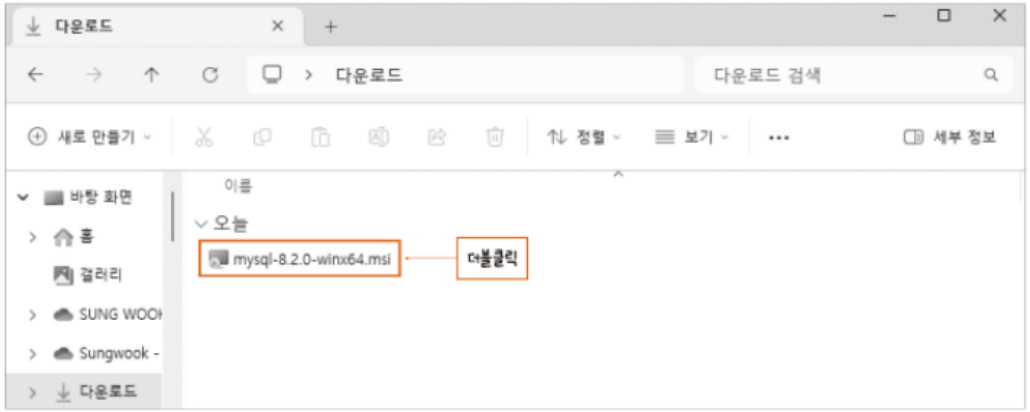
Please note that these are old versions. New releases will have recent bug fixes and features!
To download the latest release of MySQL Community Server, please visit MySQL Downloads.

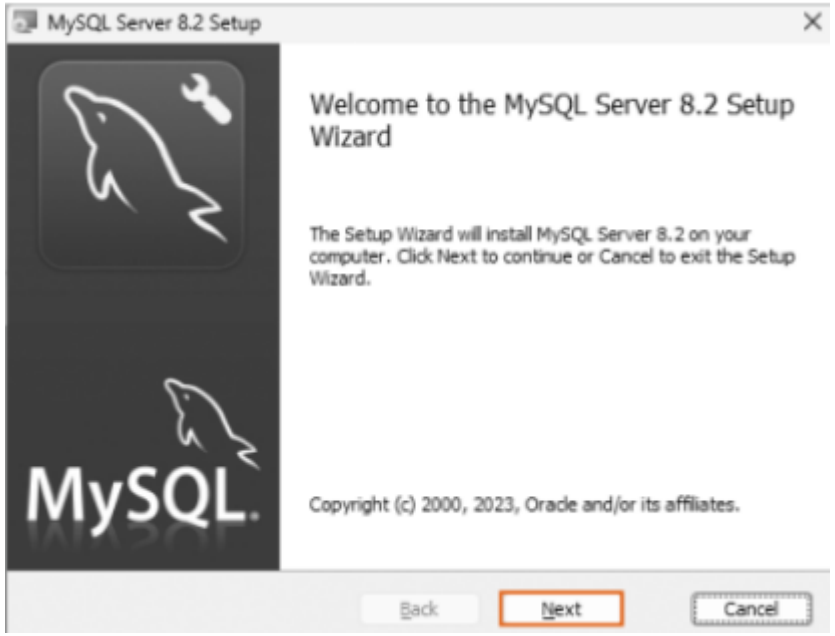
Product Version: 8.2.0
Operating System: Microsoft Windows

Windows (x86, 64-bit), MSI Installer	Oct 13, 2023	130.4M	Download
(mysql-8.2.0-winx64.msi)		MD5: 74654eaae0e0cd8f1329f8cc91279f63 | Signature	
Windows (x86, 64-bit), ZIP Archive	Oct 12, 2023	241.5M	Download
(mysql-8.2.0-winx64.zip)		MD5: e0b9ac00cf136a40020e579a33d080db | Signature	
Windows (x86, 64-bit), ZIP Archive **Debug Binaries & Test Suite**	Oct 12, 2023	683.5M	Download
(mysql-8.2.0-winx64-debug-test.zip)		MD5: 973451e4c2c3cdab210d24d64bfb1d31 | Signature	

실습 환경 구축하기 – MySQL 설치

1. 내려 받기가 완료되면 설치 파일을 실행

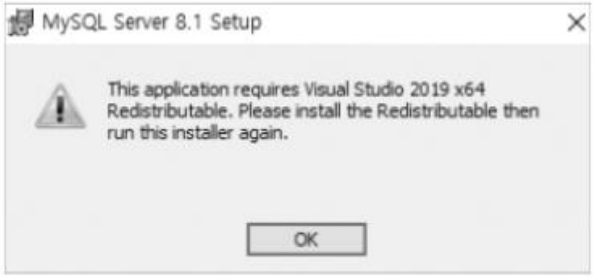




실습 환경 구축하기 – MySQL 설치

1. 내려 받기가 완료되면 설치 파일을 실행

→ 'The application requires Visual Studio..'로 시작하는 경고 메시지가 나타나는 경우



→ 윈도우에서 MySQL을 설치할 때 필요한 패키지가 설치되어 있지 않기 때문에 발생

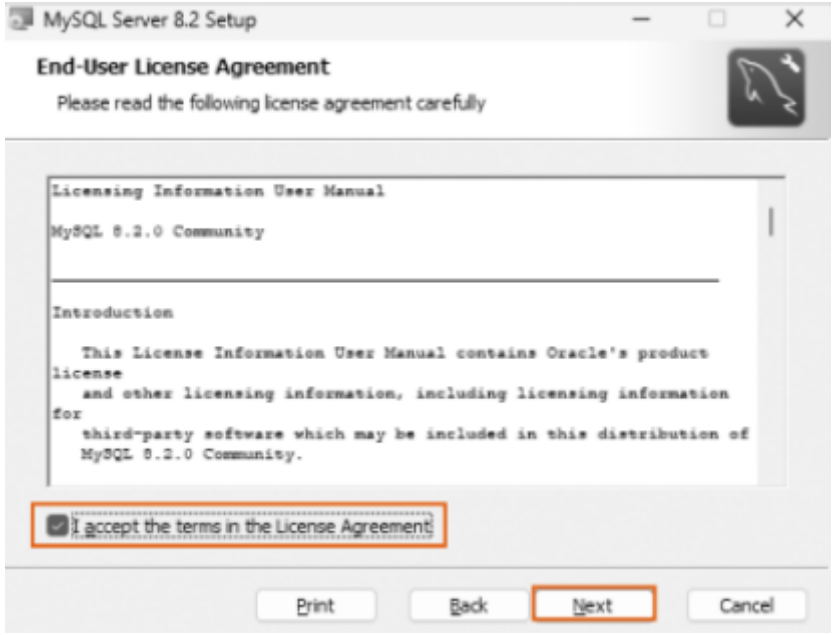
→ 최신 Visual C++ 재배포 가능 패키지 설치

(https://learn.microsoft.com/en-us/cpp/windows/latest-supported-vc-redist?view=msvc-170)

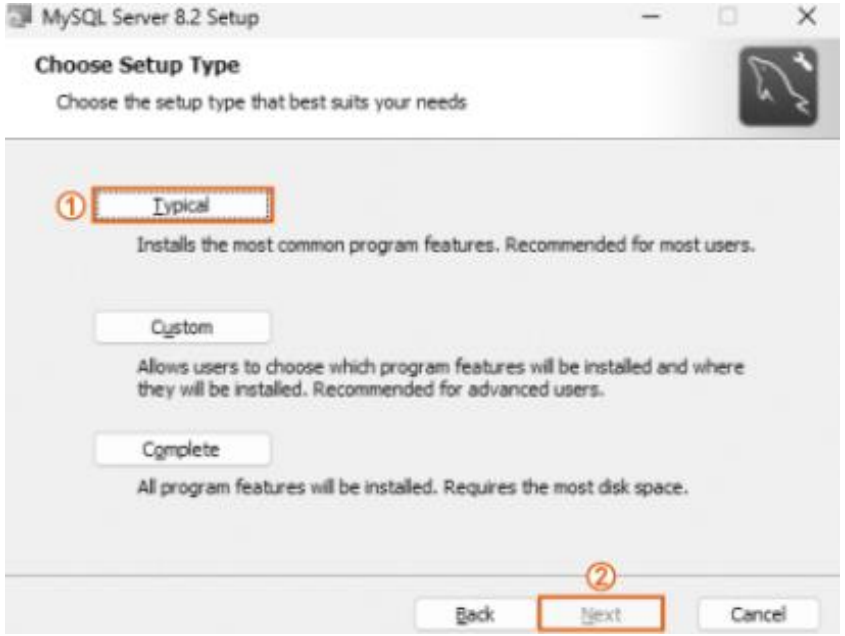
Architecture	Link	Notes
ARM64	https://aka.ms/vs/17/release/vc_redist.arm64.exe	Permalink for latest supported ARM64 version
X86	https://aka.ms/vs/17/release/vc_redist.x86.exe	Permalink for latest supported x86 version
X64	https://aka.ms/vs/17/release/vc_redist.x64.exe	Permalink for latest supported x64 version. The X64 Redistributable package contains both ARM64 and X64 binaries. This package makes it easy to install required Visual C++ ARM64 binaries when the X64 Redistributable is installed on an ARM64 device.

실습 환경 구축하기 – MySQL 설치

2. 사용자 라이선스 정책 화면에서 라이선스에 동의

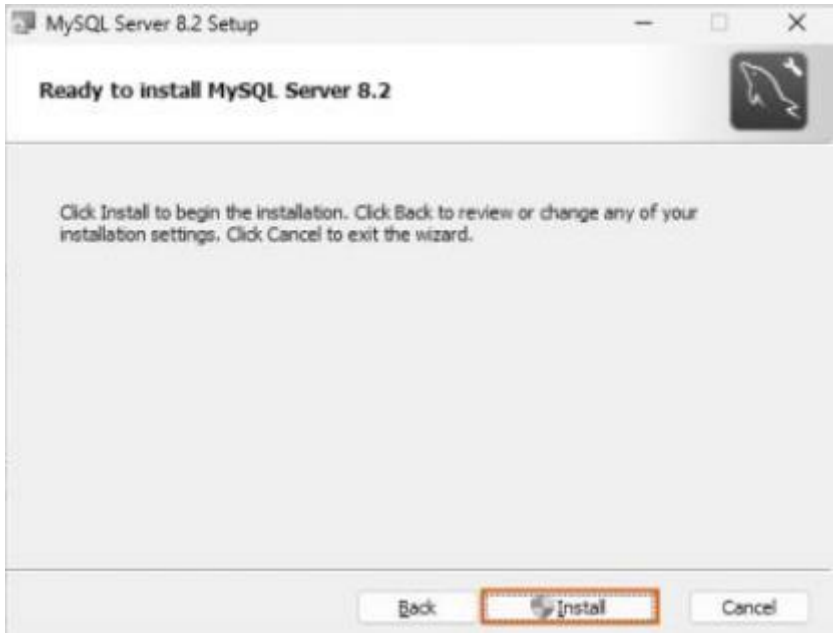


3. [Typicall]을 선택, 그 다음 [Next] 클릭

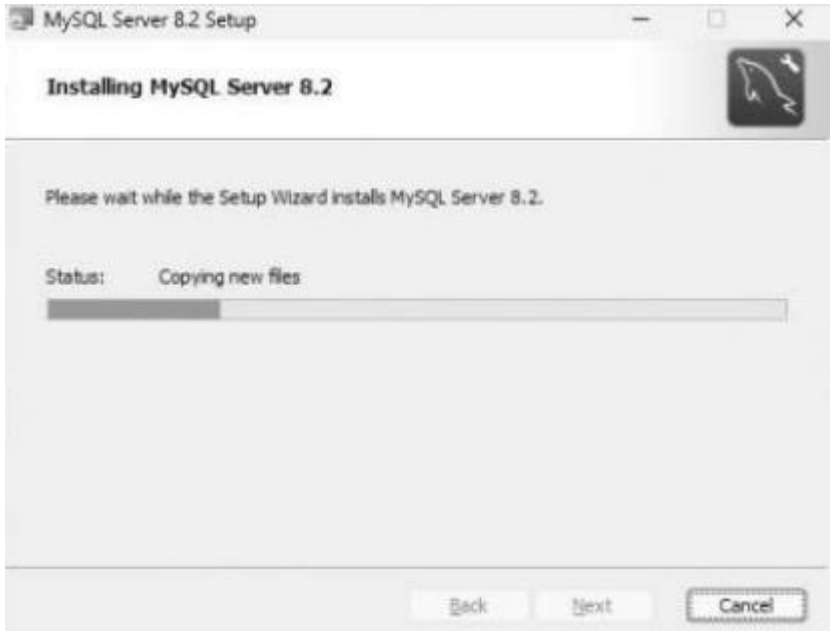


실습 환경 구축하기 – MySQL 설치

4. [Install] 클릭

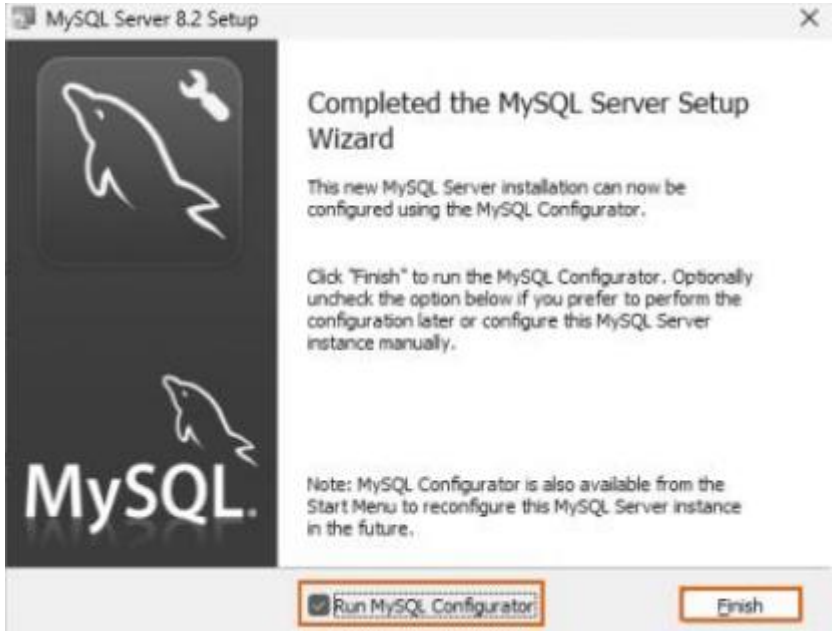


5. 설치 진행 과정이 표시됨



실습 환경 구축하기 – MySQL 설치

6. MySQL 설치 완료, MySQL을 사용하기 위해 몇 가지 구성 설정, [Run MySQL Configurator]을 선택 후 [Finish] 버튼 클릭



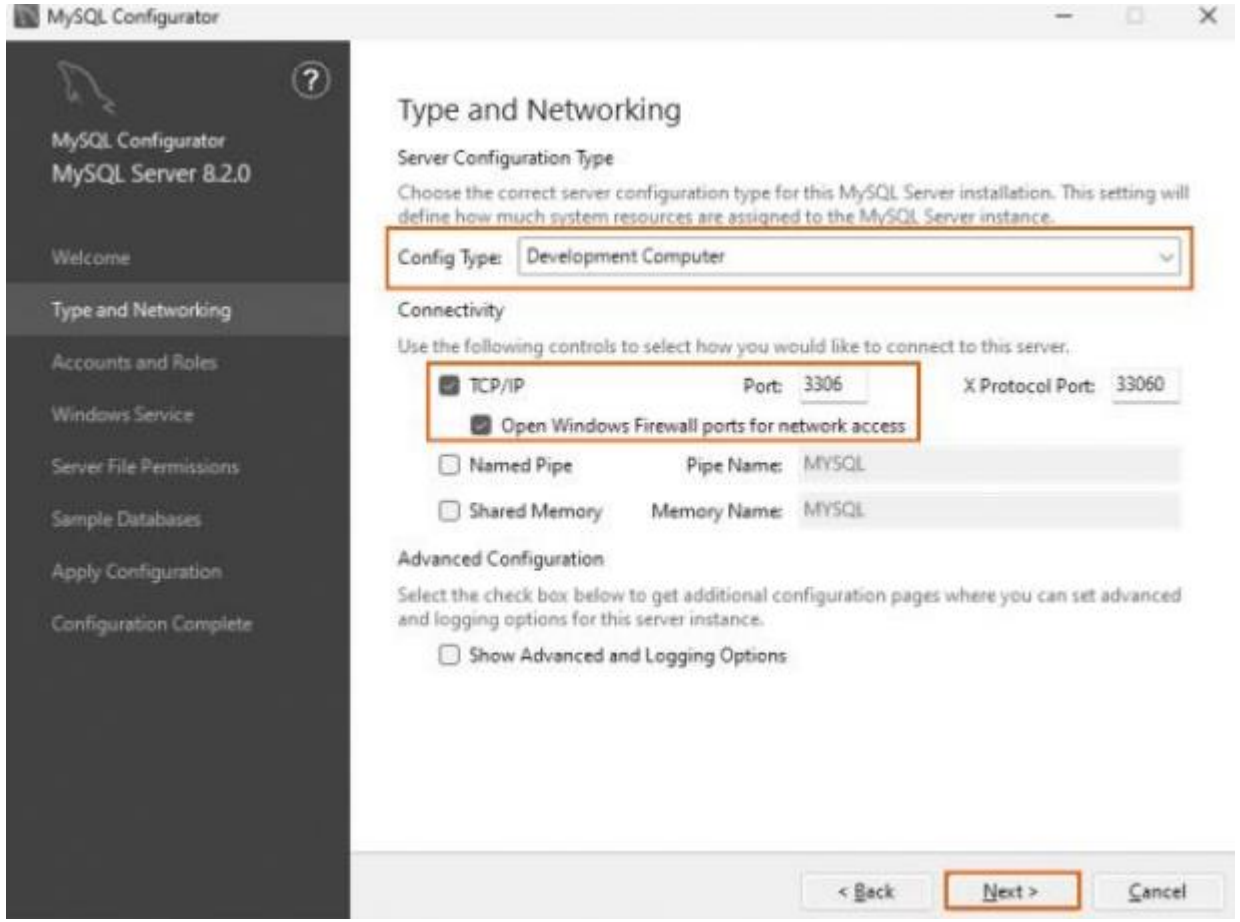
실습 환경 구축하기 – MySQL 설치

7. MySQL Configurator 창이 나타나면 [Next] 클릭



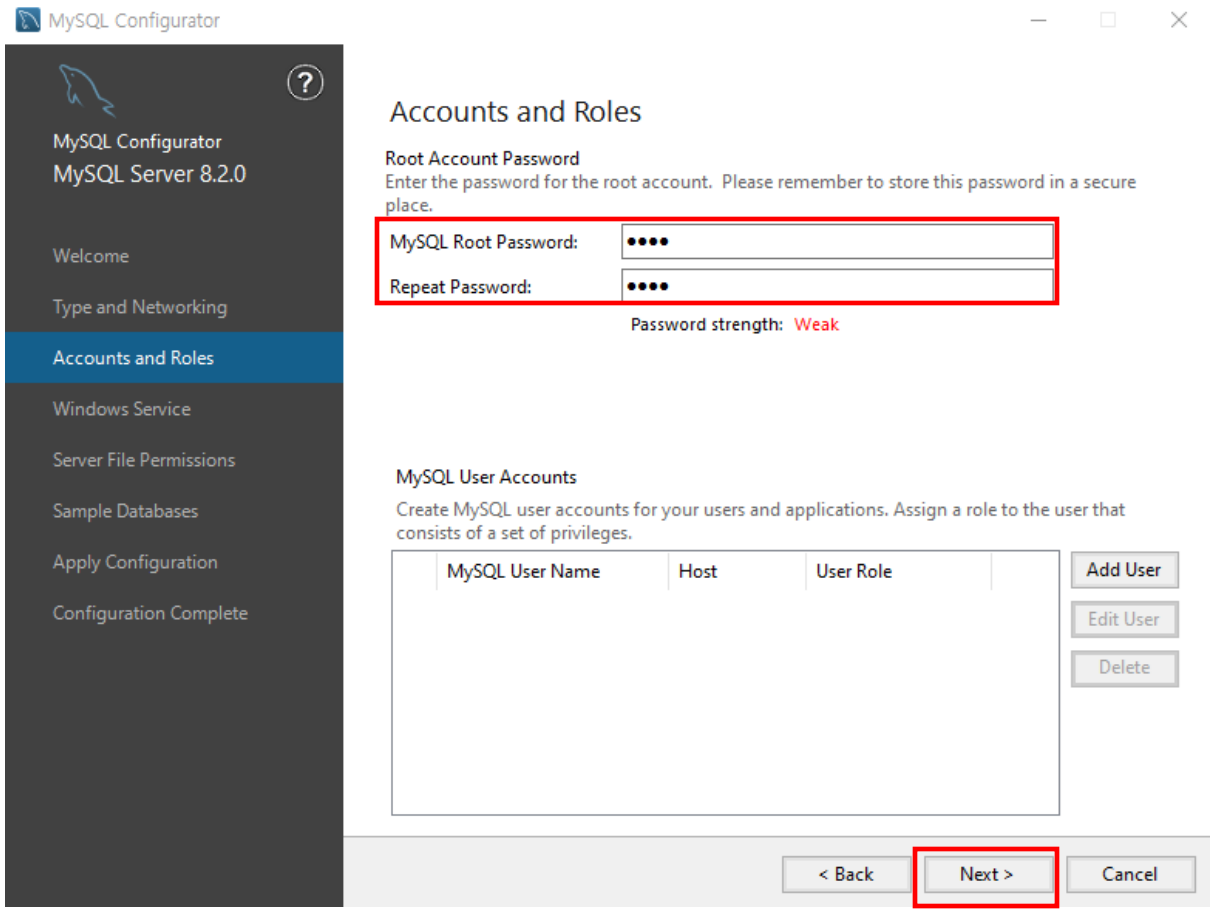
실습 환경 구축하기 – MySQL 설치

8. 'Config Type' 항목을 찾아 [Development Computer]로 선택, 'Connectivity' 항목에서는 'Port'가 3306으로 되어 있는지 확인, 확인 후,[Next]를 클릭한



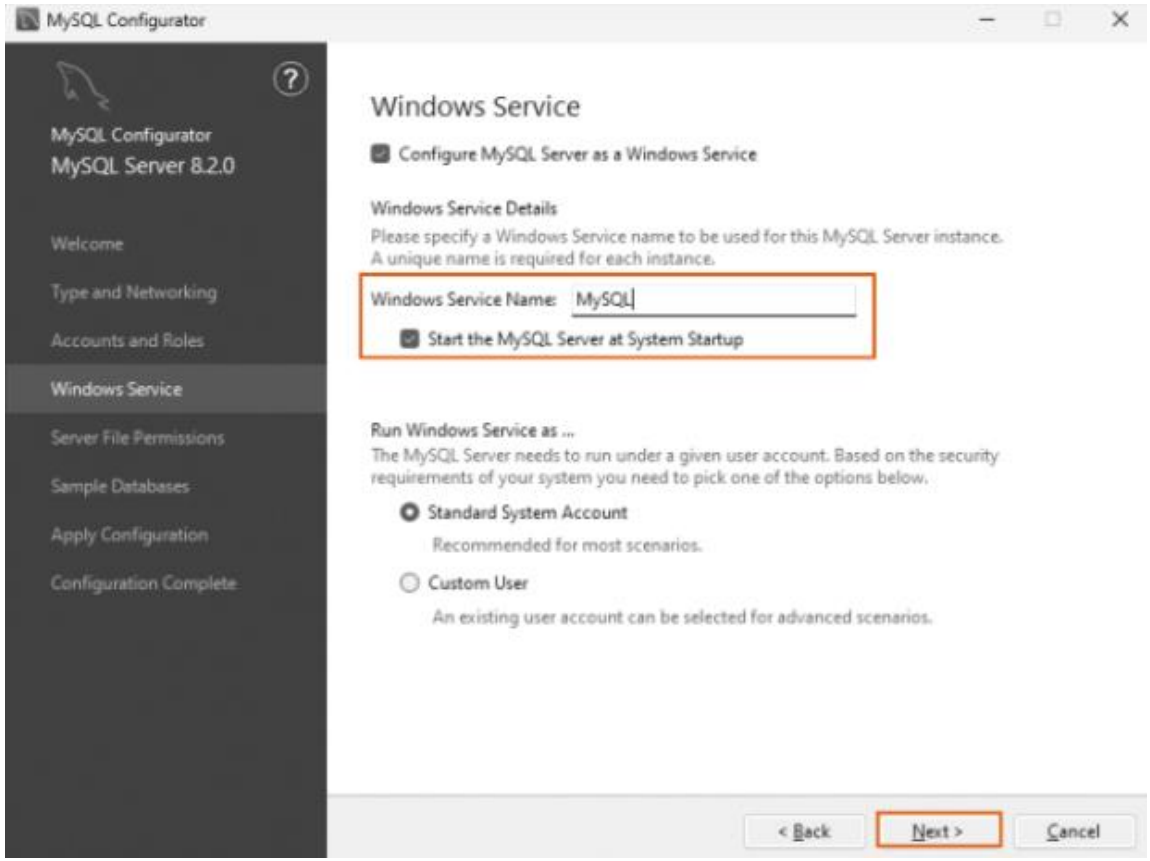
실습 환경 구축하기 – MySQL 설치

9. MySQL 관리자(루트 root)의 비밀번호를 1234로 설정



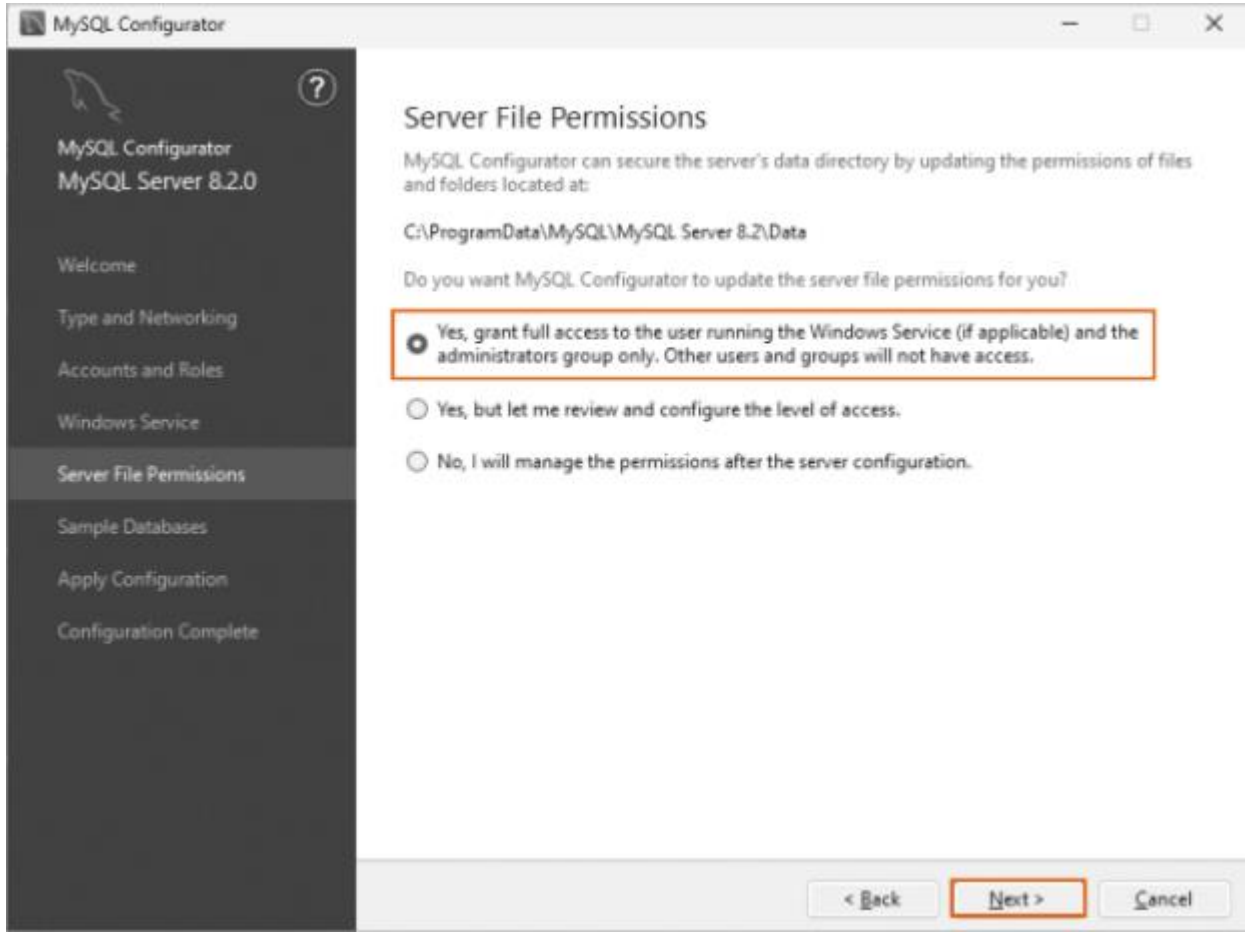
실습 환경 구축하기 – MySQL 설치

10. Windows Service 단계에서는 MySQL 서버를 윈도우 서비스에 등록하기 위한 설정을 진행, Windows Service Name 항목에 'MySQL82'으로 자동 입력되어 있는데, 일반적으로 많이 사용하는 이름으로 서비스를 등록할 수 있도록 'MySQL'로 수정, 나머지 설정은 그대로 유지하고 [Next] 클릭



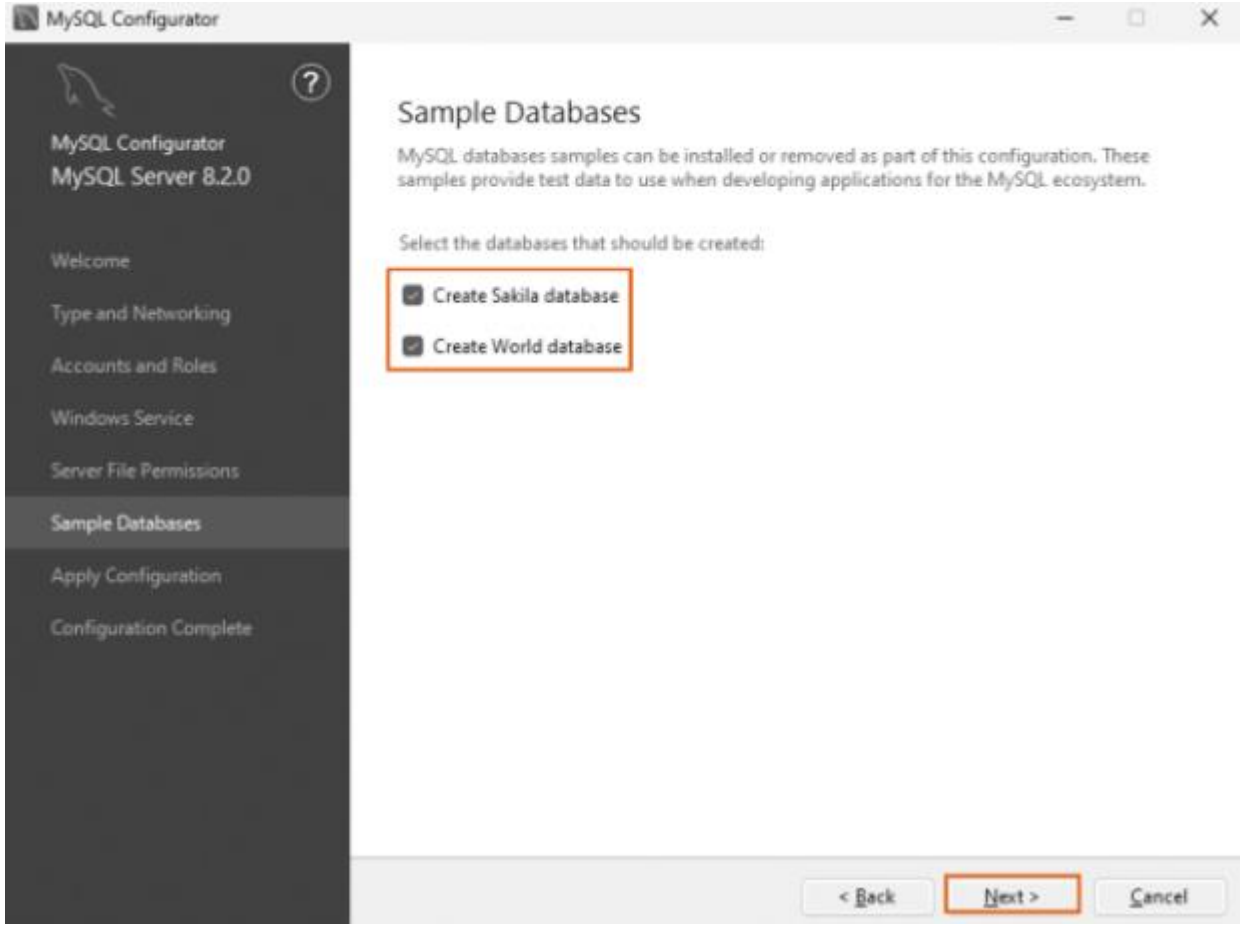
실습 환경 구축하기 – MySQL 설치

11. Server File Permission 단계에서는 기본 설정을 그대로 유지하고 [Next] 클릭



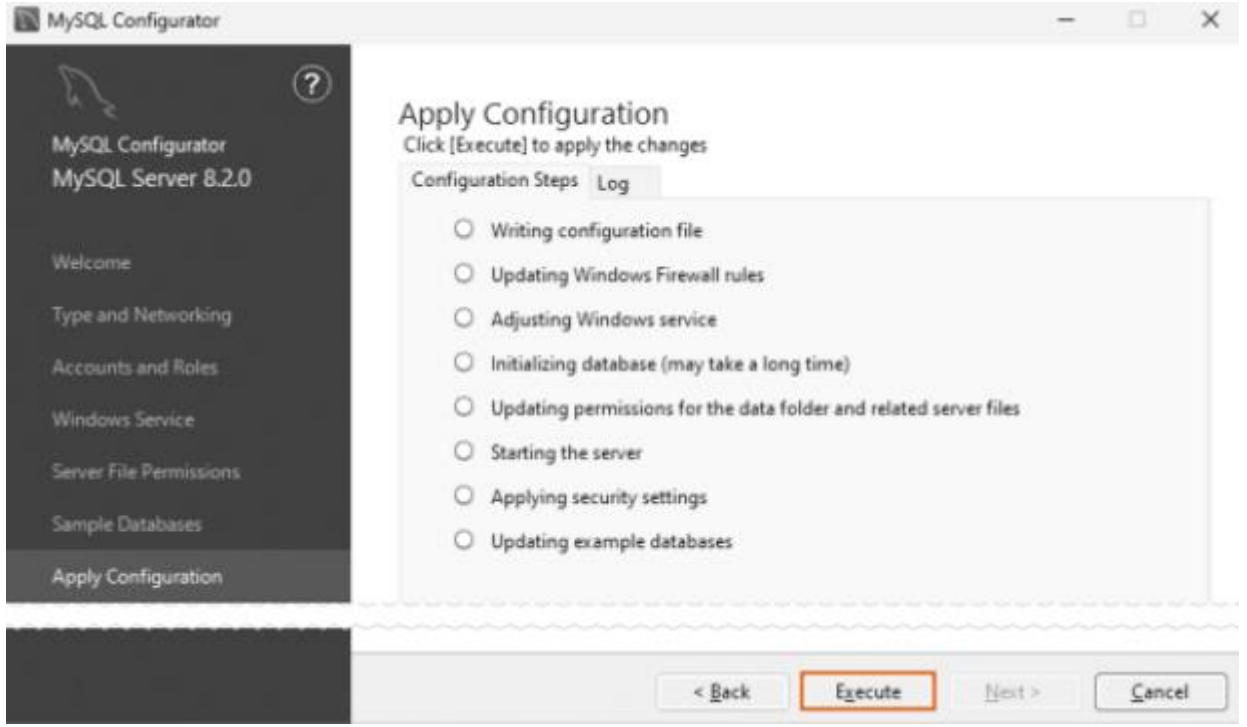
실습 환경 구축하기 – MySQL 설치

12. Sample Databases 단계에서는 샘플 데이터베이스를 반드시 선택하고 [Next] 클릭



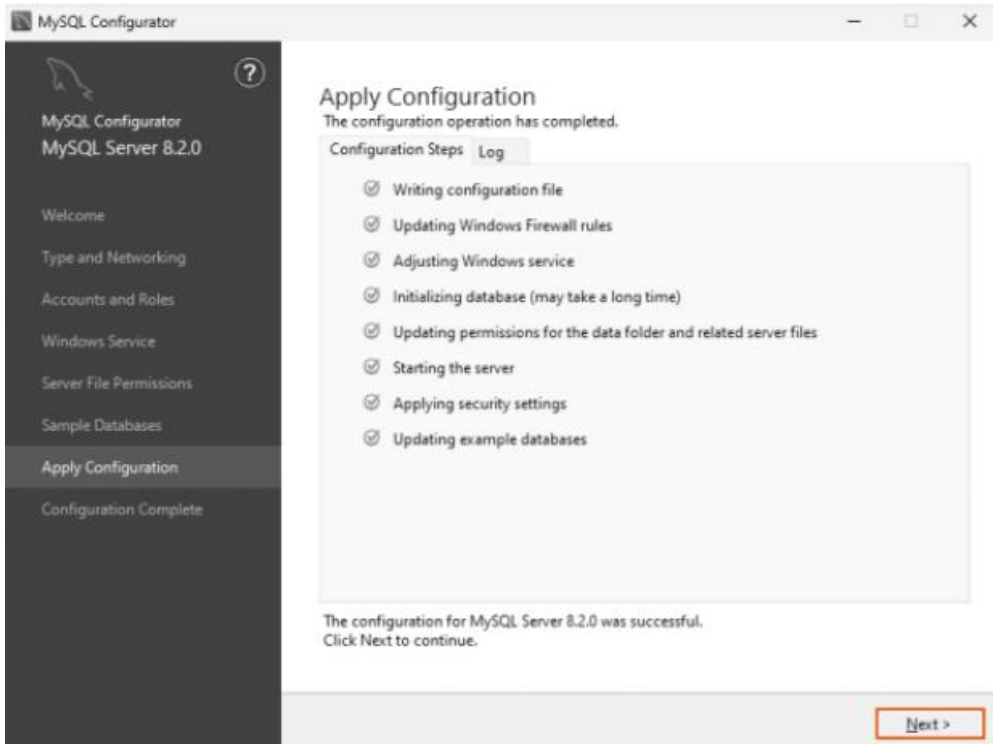
실습 환경 구축하기 – MySQL 설치

13. Apply Configuration 단계는 지금까지 설정한 구성을 적용하는 단계, [Execute]를 클릭하여 설정 적용

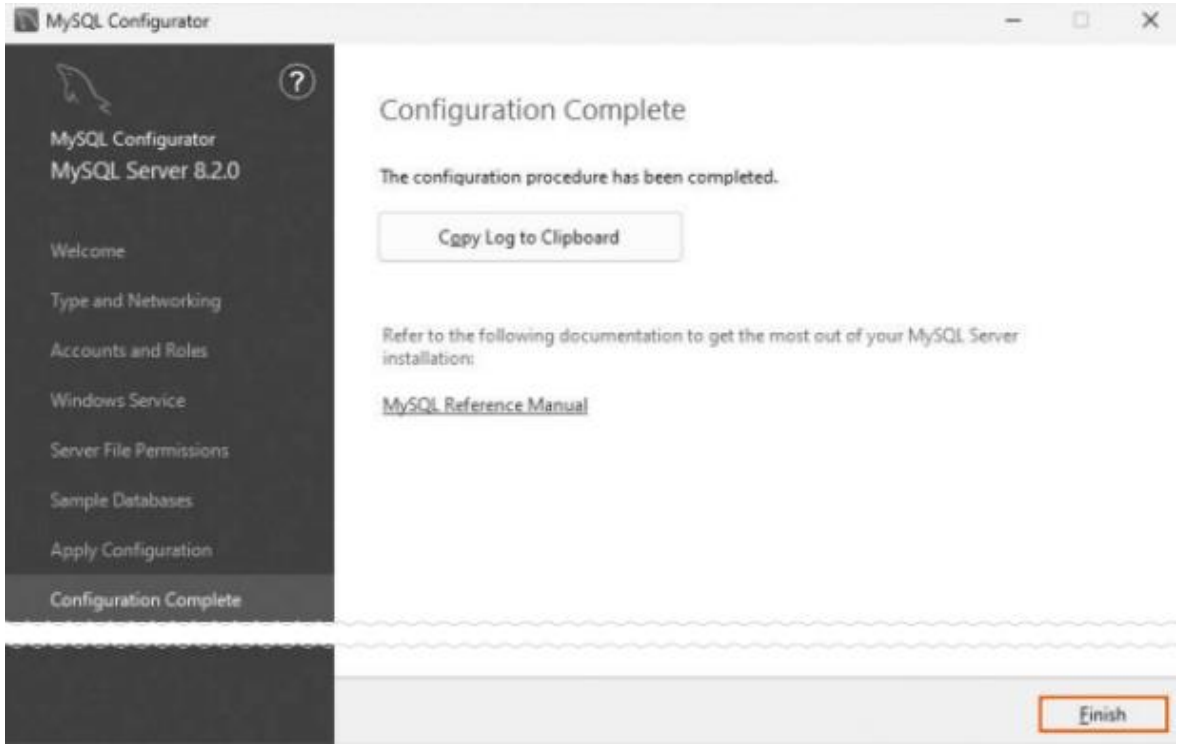


실습 환경 구축하기 – MySQL 설치

14. 각 항목 앞에 녹색 체크가 표시되면 [Next] 클릭

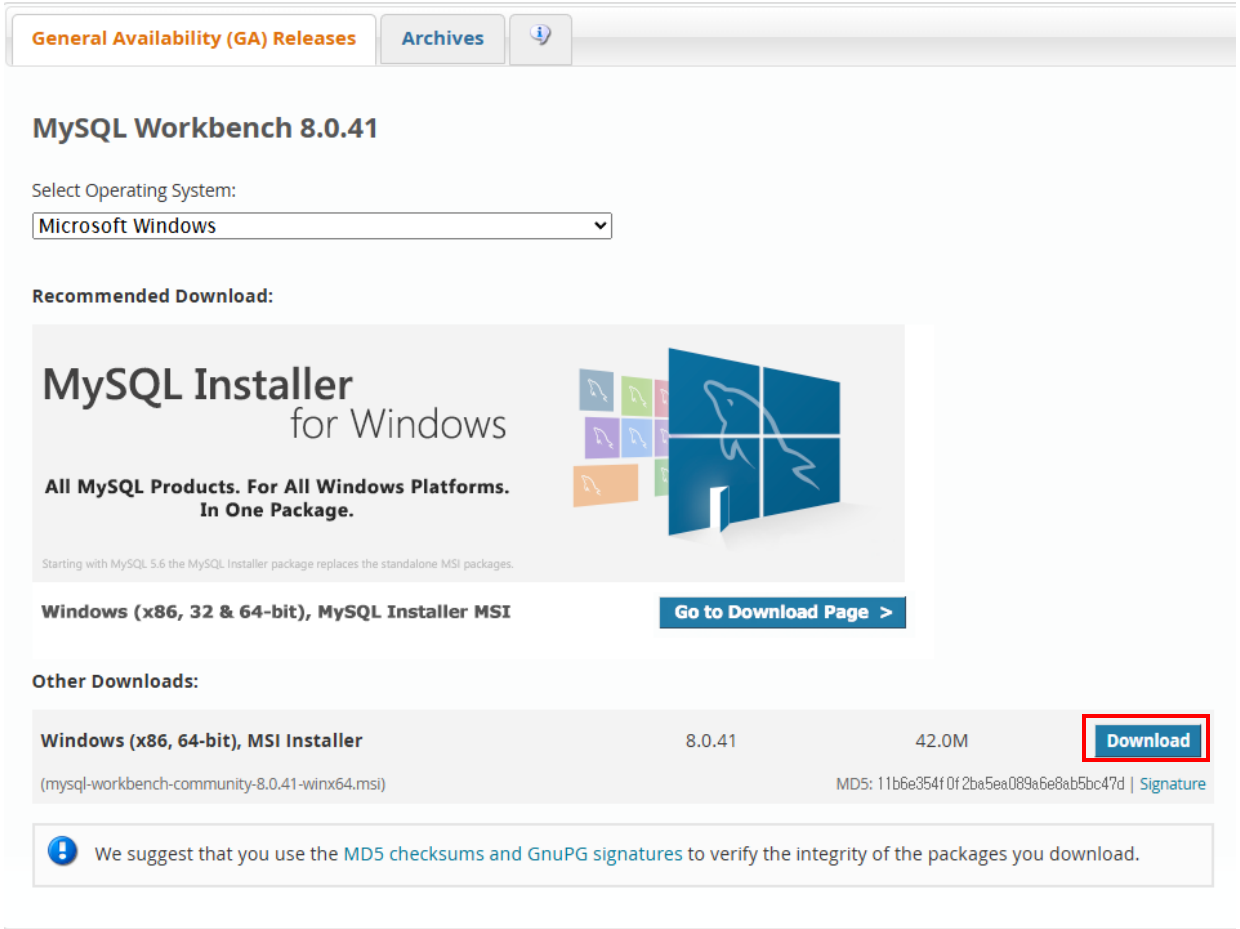


15. 모든 설정 완료, [Finish] 클릭

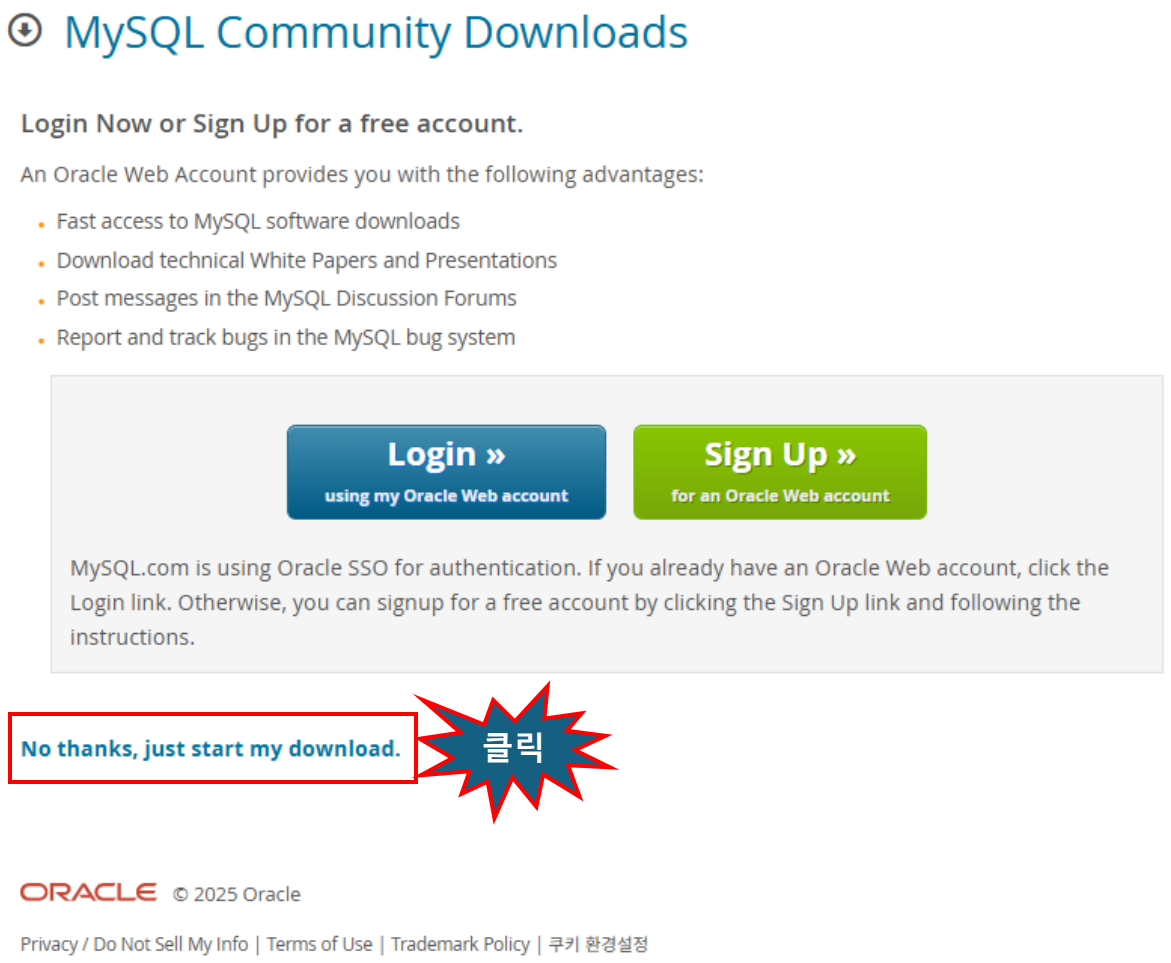


실습 환경 구축하기 – MySQL 워크벤치 설치

- MySQL 워크벤치 : MySQL 데이터베이스를 효율적으로 설계, 개발, 관리할 수 있도록 지원하는 소프트웨어
- MySQL 워크벤치 다운로드(https://dev.mysql.com/downloads/workbench/)

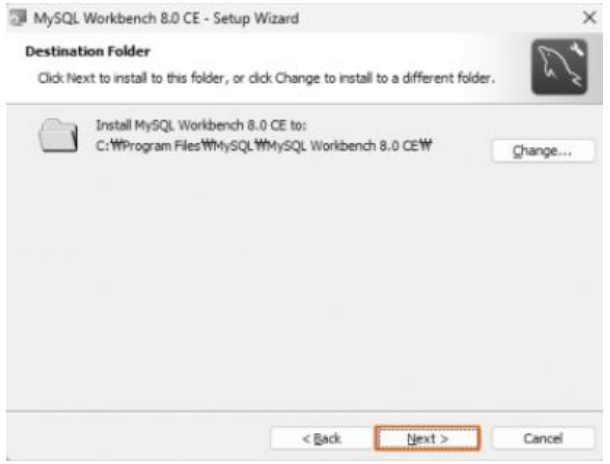


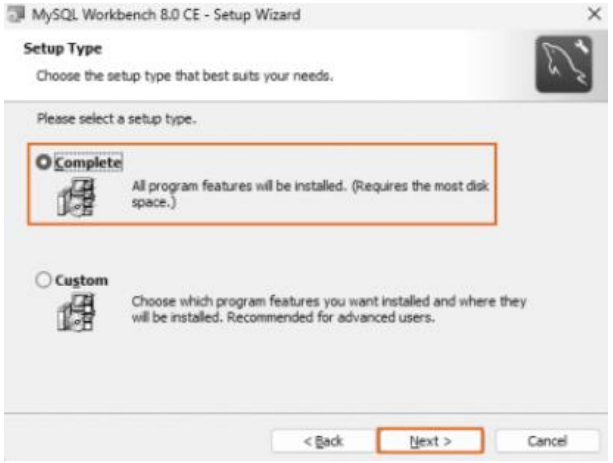
실습 환경 구축하기 – MySQL 워크벤치 설치

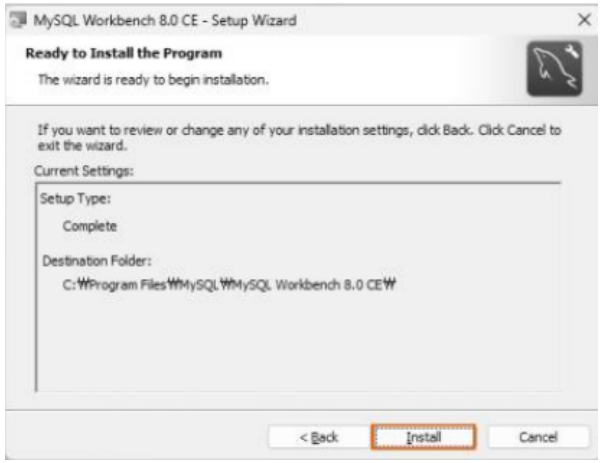


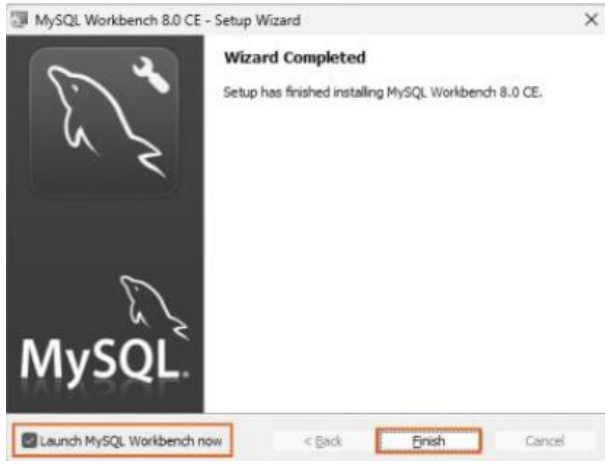
실습 환경 구축하기 – MySQL 워크벤치 설치











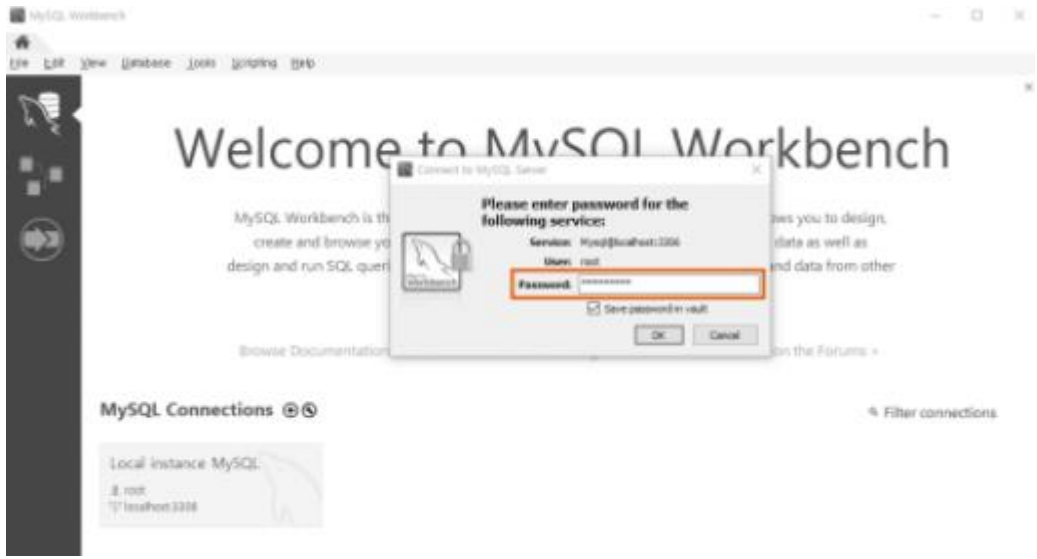
실습 환경 구축하기 – MySQL 워크벤치 실행

1. 처음 MySQL 워크벤치를 실행하면 다음과 같은 화면이 등장한다. MySQL Connections 목록에 Localhost에 대한 커넥션이 root 계정으로 자동 생성되어 있다. 해당 커넥션을 클릭하여 MySQL에 연결을 시도한다



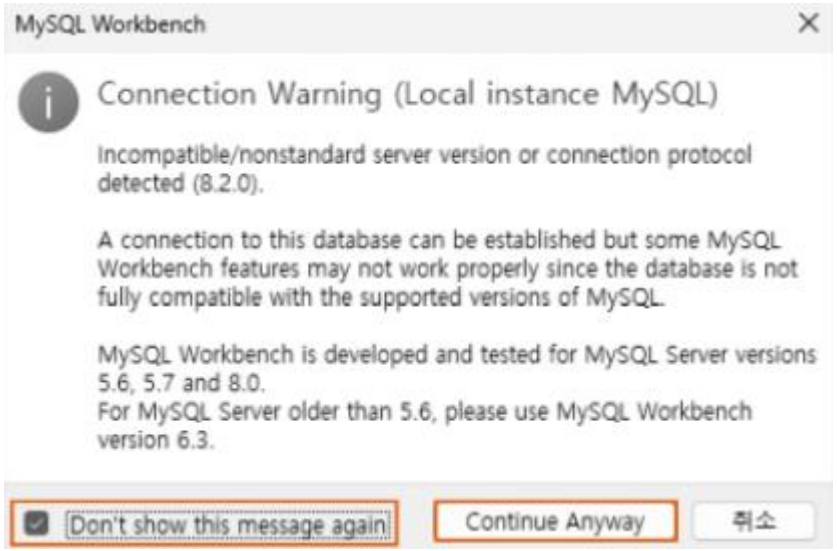
실습 환경 구축하기 – MySQL 워크벤치 실행

2. root 계정의 비밀번호를 입력하는 팝업 창이 나타나면 MySQL을 설치할 때 입력한 비밀번호(1234)를 입력한다



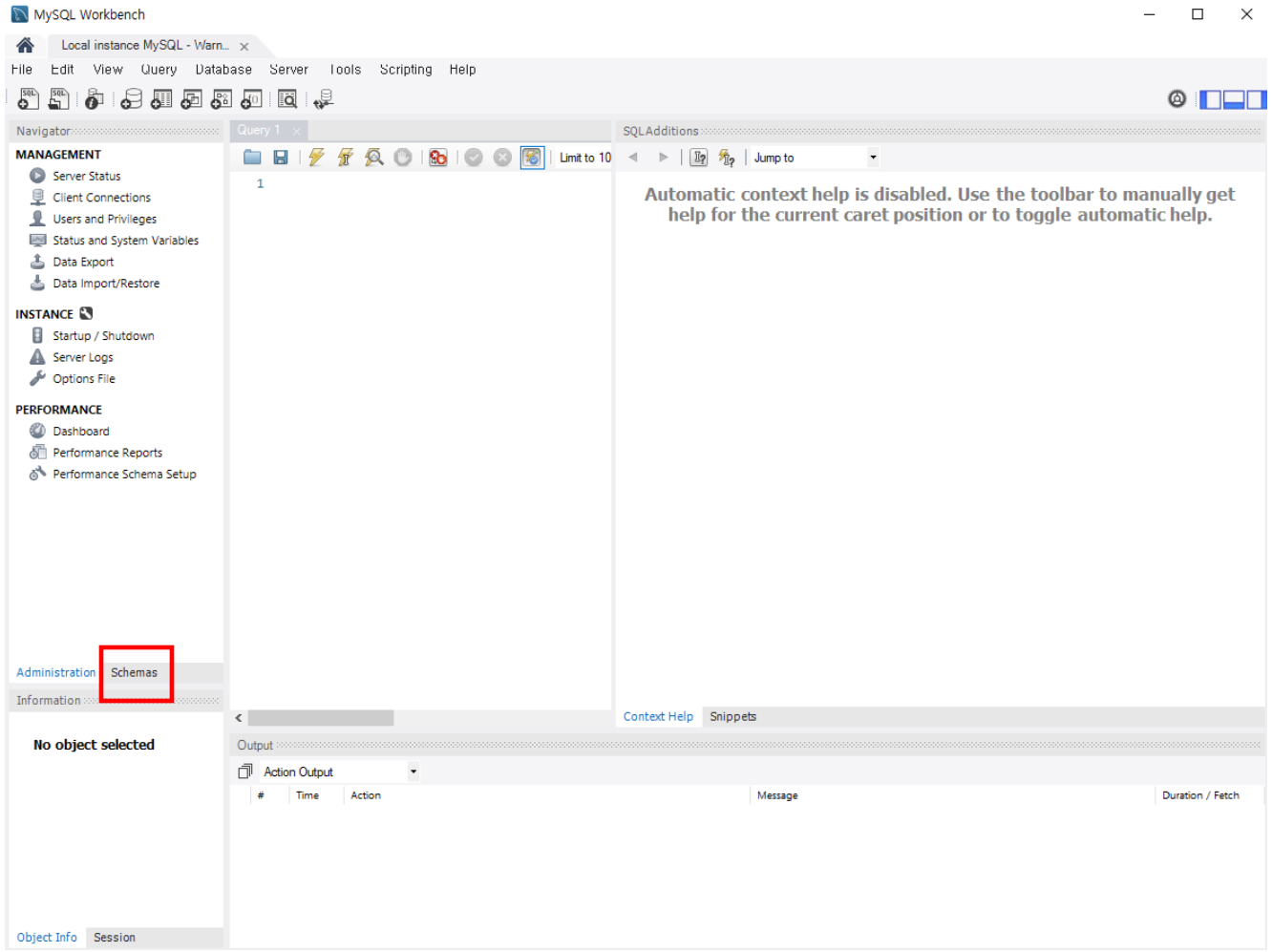
실습 환경 구축하기 – MySQL 워크벤치 실행

3. 커넥션이 처음 연결되면 다음과 같은 팝업 안내 창이 나타난다. [Don't show this message again]을 선택하고 [Continue Anyway]를 클릭한다.



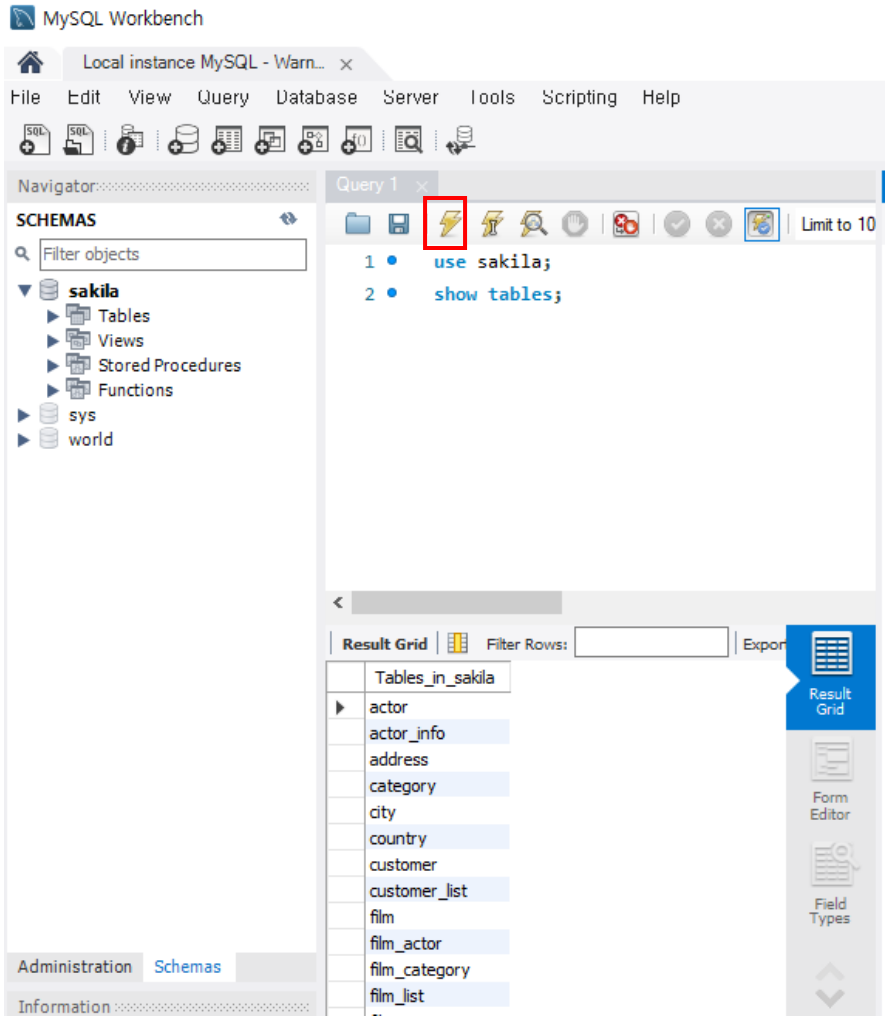
실습 환경 구축하기 – MySQL 워크벤치 실행

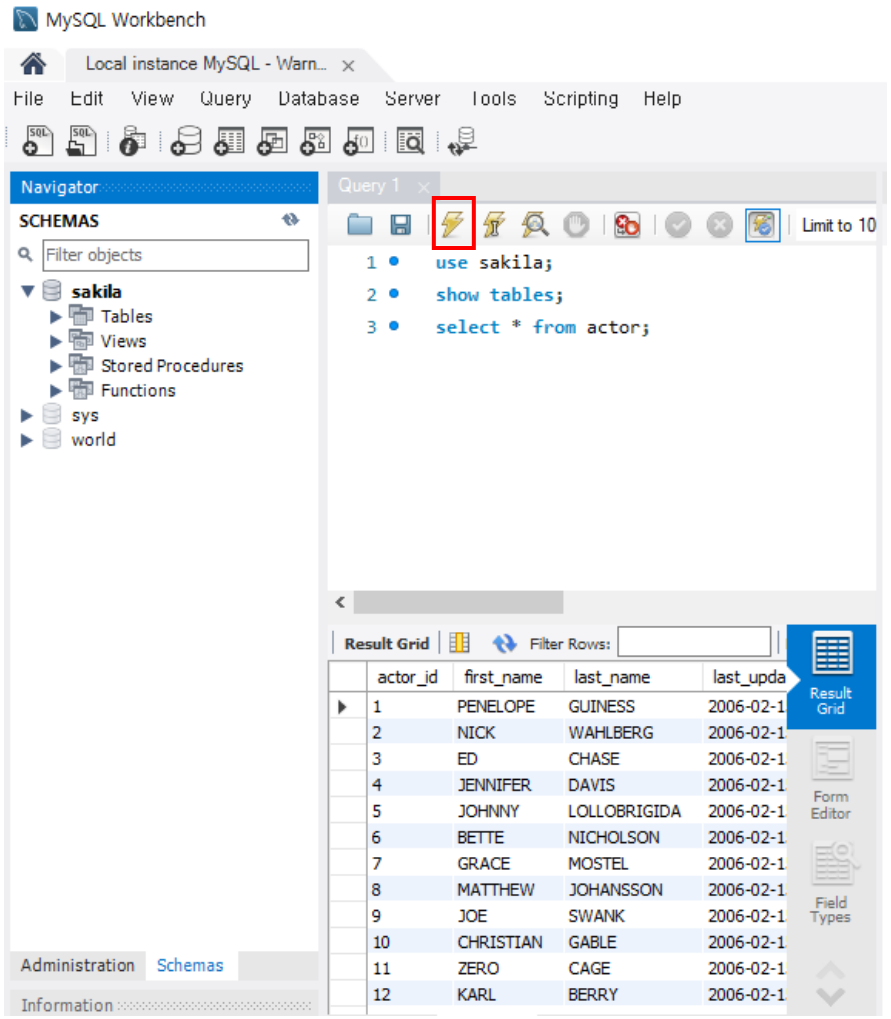
4. MySQL에 접속했으면 다음과 같이 Schemas를 클릭한다



실습 환경 구축하기 – MySQL 워크벤치 실행

5. 다음과 같이 쿼리를 작성하고 실행하여 정상적으로 작동하는지 확인한다.





02. Python

python은 데이터를 처리하는 프로그래밍 언어이다.

데이터	값
숫자	1, 0.5
문자	'가',"가", 'a', "a", '1', "1"
문자열	'I am a boy', "행복하세요 "
숫자, 문자의 집합	[1,3,5], ('a', 'abc', '가나다'), …

python은 데이터를 처리하는 프로그래밍 언어이다.

- 자료형(Data Type)
  1) 정의: 컴퓨터가 처리할 수 있는 데이터의 '종류'와 '형태'를 규정한 것
  2) 자료형의 종류
     ① 기본 자료형: 단일 값을 저장
     ② 묶음 자료형: 여러 값의 묶음

python은 데이터를 처리하는 프로그래밍 언어이다.

- 기본 자료형

자료형	파이썬 타입	설명	실생활 비유
숫자형	int, float	계산을 위한 숫자 (정수, 실수)	영수증 금액, 나이
문자열	str	따옴표(' ', " ")로 감싸진 글자	이름, 주소, 이메일
불리언	bool	참(True)과 거짓(False)의 논리 값	로그인 여부 (Y/N)

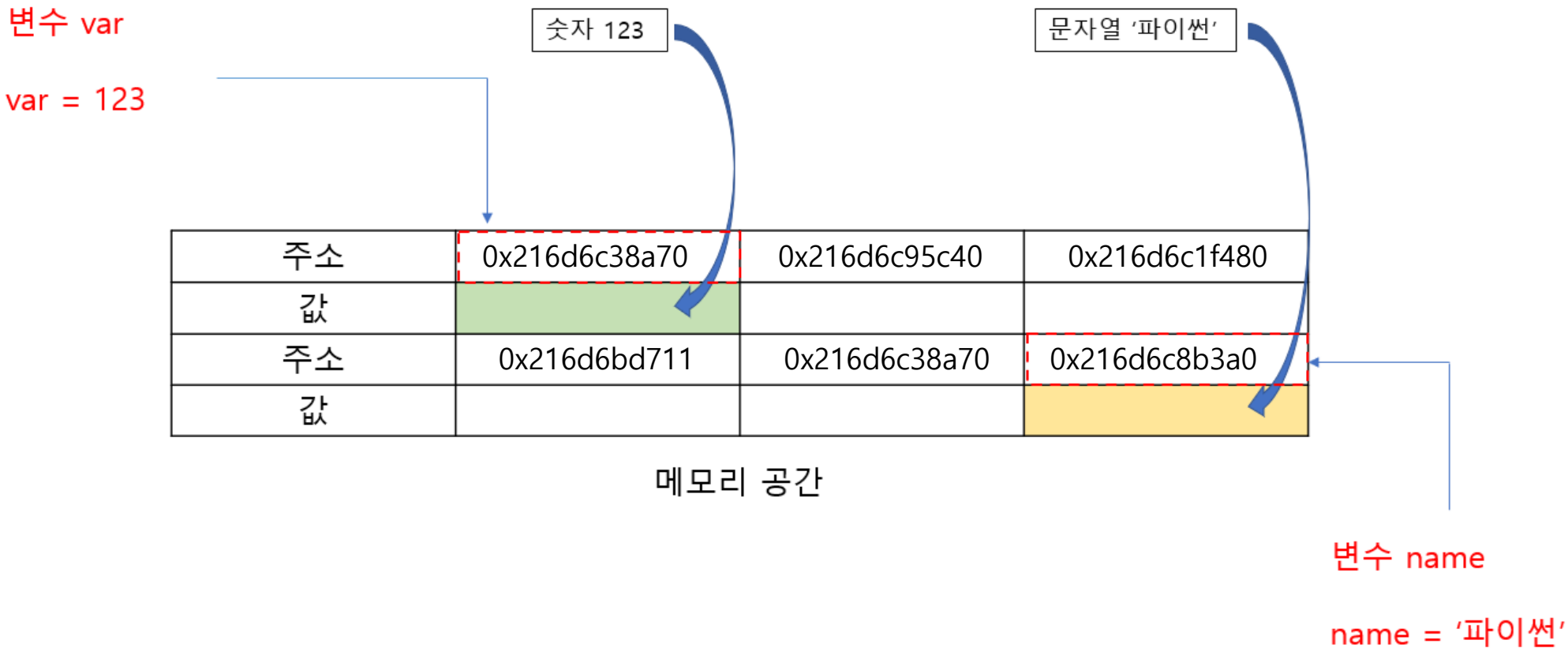
python은 데이터를 처리하는 프로그래밍 언어이다.

- 묶음 자료형

자료형	파이썬 타입	기호	핵심 특징	실생활 비유
리스트	list	[ ]	• 순서가 (인덱스) 있음 • 데이터를 언제든 추가, 삭제, 수정할 수 있는 가장 대표적인 묶음	마트 장바구니 목록(물건을 계속 넣고 뺄 수 있음)
튜플	tuple	( )	• 순서가 (인덱스) 있음. • 한 번 만들면 절대 내용을 바꿀 수 없는(Read-Only) 안전한 묶음입니다.	스마트폰 GPS 위도·경도(바뀌면 안 되는 고정된 값)
딕셔너리	dict	{ }	• 순서 대신 키(Key)와 값(Value)을 쌍으로 묶어 저장 • 키를 통해 원하는 값 추출 가능	휴대폰 연락처(이름 → 전화번호)

변수와 데이터

- 변수 : 데이터(값)가/이 저장된 공간의 이름



컴퓨터를 통한 데이터의 입력과 출력

- 입력 : 키보드
- 출력 : 모니터

```
a = 1


print(a)
```

데이터 출력하기

- 기본 출력 함수 : print(data)
  1) '숫자' 데이터 출력 : print(숫자)
     예시) print(2026)
  2) '문자열' 데이터 출력 : print("표현할 문자열") / print('표현할 문자열')
     예시) print('Hello python') / print("안녕 파이썬")

데이터 출력하기

- f 접두사 (f-string)를 이용한 문자열 출력

▶ 문자열 안에 변수를 직접 삽입하여 변수에 저장된 데이터를 문자열과 함께 출력하는 방식

예시1)

```
height = 175
print(f'저의 키는 {height}cm 입니다.')
```

예시2)

```
name = 'GD'
print(f'제 이름은 {name}입니다.')
```

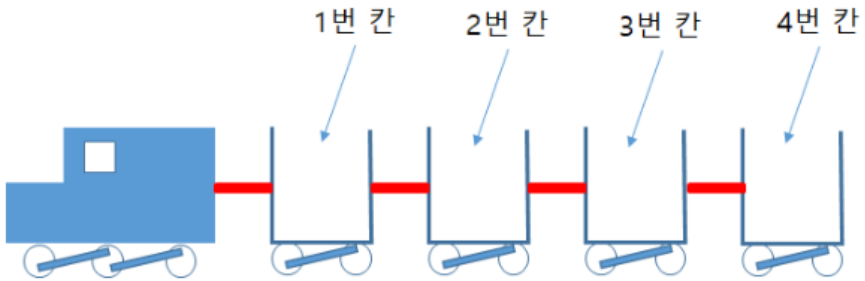
변수 네이밍 규칙

- 사용 가능한 문자
  1) 소문자(a~z)
  2) 대문자(A~Z)
  3) 숫자(0~9)
  4) 한글도 가능 → 권장 사항(X)
  5) 특수기호는 언더바(_) 만 허용

- 규칙
  1) 대소 문자를 구분
  2) 숫자는 맨 처음에 올 수 없다.
  3) 변수의 이름 사이의 공백 불가능(a b = 10)

리스트(list)

- python의 순서형 묶음 자료형
  1) 여러 개의 데이터들을 저장하고, 사용하는 방법을 정의
  2) 특징
     ① 전체 데이터가 하나의 묶음
     ② 내부적으로 여러 개의 데이터를 저장할 수 있도록 여러 개의 변수를 포함



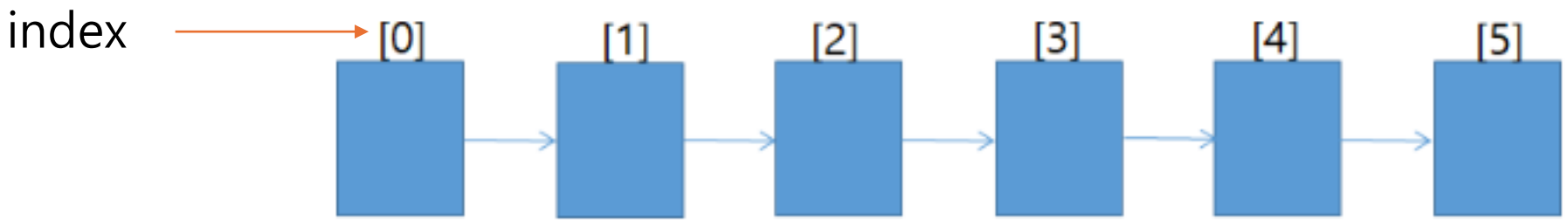
     ③ 리스트 내부의 성분 원소들은 콤마(,)으로 구분
     ④ 원소들 간에 순서가 있으며, 추가, 수정, 삭제 가능

리스트(list) 생성

변수 = [원소1, 원소2, 원소3, ...]

```
a = [ ]
b = [1, 2, 3]
c = [‘My’, ‘name', ‘is', ‘MH']
d = [1, 2, ‘My', ‘name']
e = [1, 2, [‘My', ‘name']]
```

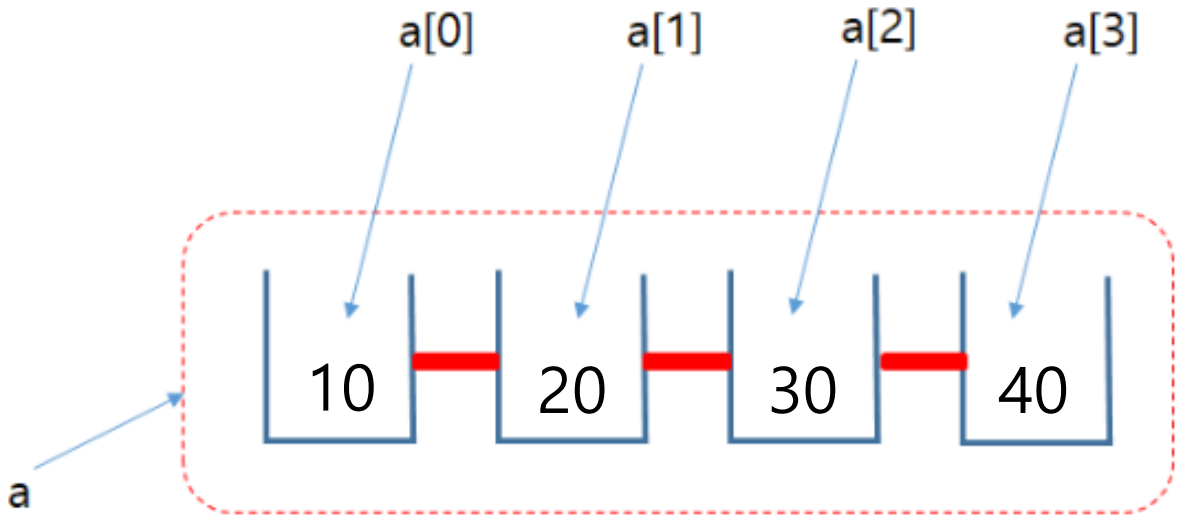
성분 원소들에 대한 접근



인덱싱(indexing)

리스트[인덱스]

- 결과 : 인덱스에 위치한 값 추출



인덱싱(indexing) 실습

```
list1 = [1, 2, 3, ['a', 'b', 'c']]
```

- list1에서 문자 'b'만 뽑아서 출력하시오.

슬라이싱(slicing)

리스트[start 인덱스 : stop 인덱스]

- start 인덱스부터 stop 인덱스 바로 전까지 값 추출 (start <= x < stop)

```
list2 = [0, 10, 20, 30, 40]

list2[0:3]
```

```
[0, 10, 20]
```

```
list2[3:]
```

```
[30, 40]
```

```
list2[3:4]
```

```
[30]
```

딕셔너리(dictionary)

- python의 매핑형 묶음 자료형

  1) 매핑(mapping) = 1:1 대응

  2) dictionary는 단어 그대로 해석하면 사전이라는 뜻, 단어와 의미가 1:1로 대응 → 1쌍(pair)

단어	의미
person	사람

- 특징

  1) data → Key와 Value가 1:1로 대응 → key : value (1쌍)

  2) key : 중복 불가능, value : 중복 가능

  3) index(X) → **key로 data에 접근**

key	value
name	이영배
job	개발자

딕셔너리(dict) 생성

변수 = {Key1 : Value1, Key2 : Value2, ... }

```
a1 = {}
b1 = { “name” : ”MH” }
c1 = { 1 : ‘송민호’, 2 : ‘화가’}
```

딕셔너리(dict) 생성 실습

key	value
name	MH
age	20
phone	010-1234-1234

[dict 정보]

값(value) 가져오기

딕셔너리 변수[Key]

key	value
name	MH
age	20
phone	010-1234-1234

데이터 추가

딕셔너리 변수[key] = value

key	value
name	MH
age	20
phone	010-1234-1234
birth	03/27

추가

[dict 정보]

데이터 추가 실습

1. 변수 dict_test를 다음과 같이 만들기

key	value
노래제목	아무노래

2. 데이터 추가를 통해 다음과 같이 정보를 저장하기

key	value
노래제목	아무노래
가수	지코
날짜	2020.01.13

특정 데이터 삭제 : 예약어 del

del 변수[index]

데이터 수정

딕셔너리 변수[key] = value

key	value
name	MH
age	20
phone	010-1234-1234
birth	03/27

수정 → 08/27

[dict 정보]

튜플(tuple)

- python의 순서형 묶음 자료형
  1) 생성 : () 이용
  2) 특징 : 리스트와 같은 개념이지만 값을 변경하는 것이 불가능

```
t = (1, 2, 3)

# 값 수정
t[1] = 5
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-10-a771a011aefa> in <cell line: 5>()
      3
      4 # 값 수정
----> 5 t[1] = 5

TypeError: 'tuple' object does not support item assignment
```

숫자형 자료형

- 수치 형태의 데이터를 표현하는 파이썬의 자료형
- 종류
  1) 정수형 (int, Integer): 소수점이 없는 수 (양의 정수, 음의 정수, 0)
     → 실생활 예시: 나이, 제품 수량, 방문자 수
  2) 실수형 (float, Floating-point): 소수점이 포함된 수
     → 실생활 예시: 키/몸무게, 시력, 환율, 할인율
- 정수형과 실수형 데이터 생성
  1) 변수에 숫자를 직접 대입하여 생성
  2) 예시

```
# 정수형(int) 데이터 생성
num1 = 123
# 실수형(float) 데이터 생성
num2 = 1.2
```

숫자형 자료형

- 숫자형 연산자
  1) 기본 사칙 연산 (+, -, *, /): 두 숫자형 데이터 간의 가감승제 연산 수행
  2) 특수 연산자 (**, //, %)
     ① 제곱 연산자 (**): x의 y제곱 계산, 예) 3의 4제곱 (3**4)
     ② 나머지 연산자 (%): 나눗셈 후 나머지만 추출
     ③ 몫 연산자 (//): 나눗셈 후 소수점 아래를 버리고 몫(정수)만 추출
- 나머지 연산자(%)를 활용한 데이터 조건 판별
  1) 짝수 / 홀수 구별
     ① 짝수: 2로 나눈 나머지가 0인 수 (n % 2 == 0)
     ② 홀수: 2로 나눈 나머지가 1인 수 (n % 2 == 1)
  2) 정수의 배수 구별
     : K의 배수: K로 나눈 나머지가 0인 수 (n % K == 0)

문자열(str) 자료형

- python의 문자 순서형 자료형
  1) 하나 이상의 연속된 문자로 구성된 데이터들을 저장하고, 사용하는 방법을 정의
  2) 문자열 생성
     ① ‘’ 또는 “” 이용

```
string1 = 'a'
string2 = 'apple'
string3 = "banana"
```

② ‘’’ string ‘’’ 또는 “””string””” ⇒ 여러 줄 문자열 생성

```
multi_line = """
this is line number1
this is line number2
this is line number3
this is line number4
"""
```

문자열(str) 자료형

- 인덱싱과 슬라이싱

string = "문자열은 인덱싱이 가능하다"

문	자	열	은		인	덱	싱	이		가	능	하	다
0	1	2	3	4	5	6	7	8	9	10	11	12	13

문자열(str) 자료형

- 문자열 연결하기 : 더하기 연산자를 이용해서 문자열을 연결할 수 있음

```
# 문자열 생성
string1 = "Good"
string2 = "Morning!"

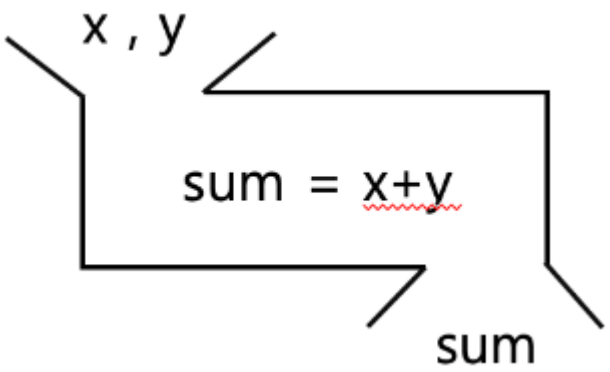
# 문자열 연결
string = string1 + " " + string2
```

블리언 자료형

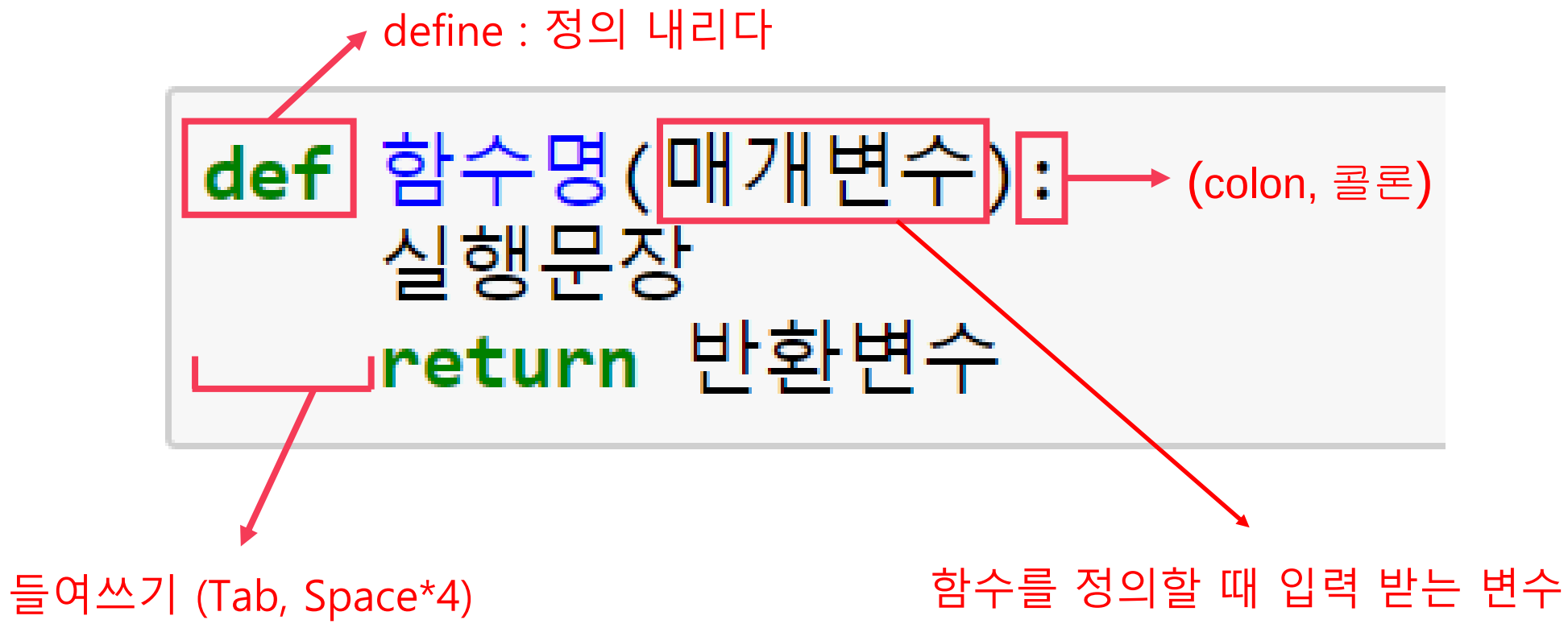
1. 값의 종류: True, False
2. 블리언 자료형의 기능: 크기 비교 시 긍정(1) 또는 부정(0) 표시
3. 블리언 자료형의 특징
   1) 문자열이지만 문자열 표시(X) → 기능이 사전에 정의된 예약어
   2) 문자열의 형태이지만 연산(+,-,*,/)이 가능
   3) 연산 시 → 문자가 아닌 수로 인식 → True(1), False(0)

함수(function)의 정의

- 입력 받은 데이터를 가지고 사전에 정의된 기능을 수행하고 결과값을 전달하는 코드의 집합

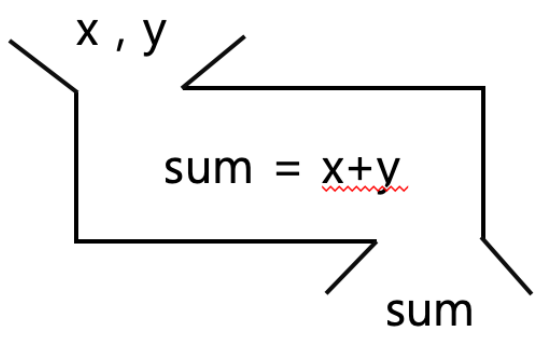


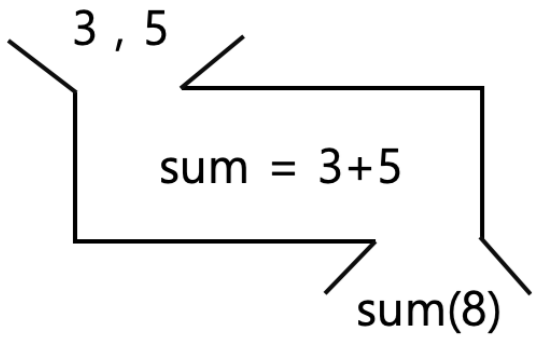
함수(function) 구조



함수(function) 호출

- 함수 실행 : 함수 이름 뒤의 소괄호 안에 필요한 인수(argument) 입력 후 실행






```
def add(x, y):
    sum = x + y
    return sum
```

```
result = add(3, 5)
print(result)
```

함수(function)의 종류

- 사용자 정의 함수
  - 사용자가 직접 정의하고 호출해서 사용
- 내장 함수
  - python에서 정의 → 호출만 하면 사용 가능
  - 자료형에 상관없이 사용 가능한 함수 예) print(), type(), len()
  - 특정 자료형에만 사용 가능한 함수
    ① 리스트 전용 함수
    ② 딕셔너리 전용 함수
    ③ 문자열 전용 함수

python 내장 함수 실습

- 리스트(list) 자료형
- 딕셔너리(dict) 자료형

리스트(list) 함수 - 값 추가

리스트.append(값)

- 맨 뒤에 값 추가

```
list5 = [0, 1, 2, 3, 4]
list5.append(5)
list5
```

```
[0, 1, 2, 3, 4, 5]
```

```
list5.append(6)
list5
```

```
[0, 1, 2, 3, 4, 5, 6]
```

list 자료 구조 - 성분 원소의 개수 확인

모든 자료형에 사용 가능한 함수 : len(리스트)

- 리스트 원소의 개수 출력

```
list13 = [0, 1, 2]
len(list13)
```

```
3
```

```
list14 = ['a','b','c','d','e','f']
len(list14)
```

```
6
```

dict 함수 - Key만 가져오기

딕셔너리 변수.keys()

```
# 딕셔너리 자료형 데이터 생성
dict_data = {'name':'박보검', 'age':31, 'phone':'010-1234-1234'}

# 전체 key 추출
keys_data = dict_data.keys()
print(f'전체 key 확인: {keys_data}')
```

```
전체 key 확인: dict_keys(['name', 'age', 'phone'])
```

dict 함수 - value만 가져오기

딕셔너리 변수.values()

```
# 딕셔너리 자료형 데이터 생성
dict_data = {'name':'박보검', 'age':31, 'phone':'010-1234-1234'}

# 전체 value 추출
values_data = dict_data.values()
print(f'전체 value 확인: {values_data}')
```

```
전체 value 확인: dict_values(['박보검', 31, '010-1234-1234'])
```

dict 함수 – key, value 쌍으로 가져오기

딕셔너리 변수.items()

```
# 딕셔너리 자료형 데이터 생성
dict_data = {'name':'박보검', 'age':31, 'phone':'010-1234-1234'}

# 전체 key, value 쌍 추출
items_data = dict_data.items()
print(f'전체 key, value 쌍 확인: {items_data}')
```

```
전체 key, value 쌍 확인: dict_items([('name', '박보검'), ('age', 31), ('phone', '010-1234-1234')])
```

dict 함수 – key로 value 가져오기

딕셔너리 변수.get('key')

```
# 딕셔너리 자료형 데이터 생성
dict_data = {'name':'박보검', 'age':31, 'phone':'010-1234-1234'}
name = dict_data.get('name')
print(name)
```

```
박보검
```

문자열(str) 함수

- capitalize() : 문자열의 첫번째 문자를 대문자로 변환
- upper() : 문자열을 대문자로 변환
- lower() : 문자열을 소문자로 변환
- title() : 문장에서 각 단어의 첫번째 문자를 대문자로 변환

```python
# 문자열 생성
string = 'python'

# capitalize() 적용
result = string.capitalize()

# 결과 확인
print(result)
```

```
Python
```

```python
# 문자열 생성
string = 'python'

# upper() 적용
result = string.upper()

# 결과 확인
print(result)
```

```
PYTHON
```

```python
# 문자열 생성
string = 'PYTHON'

# upper() 적용
result = string.lower()

# 결과 확인
print(result)
```

```
python
```

```python
# 문장 생성
string = 'just do it!'

# title() 적용
result = string.title()

# 결과 확인
print(result)
```

```
Just Do It!
```

문자열(str) 함수

- split() : 입력한 문자를 기준으로 문자열을 분리하여 리스트의 형태로 출력
- join() : 리스트의 형태로 분리된 문자열을 입력한 문자를 기준으로 연결

```
# 빈칸을 기준으로 문자열 분리하기
example = 'zero one two three'
result = 'zero one two three'.split()
print(result)

print('-'*80)

# ","을 기준으로 문자열 분리하기
example = 'python,jquery,javascript'
result = example.split(",")
print(splited)
```

```
['zero', 'one', 'two', 'three']
--------------------------------------------------------------------------------
['python', 'jquery', 'javascript']
```

```
# 리스트의 형태로 분리된 문자열 생성
colors = ['red', 'blue', 'green', 'yellow']

# 연결 시, 1칸을 띄고 연결
result = ' '.join(colors)
print(result)

print('-'*80)

# 연결 시 ", "로 연결
result = ', '.join(colors)
print(result)
```

```
red blue green yellow
--------------------------------------------------------------------------------
red, blue, green, yellow
```

문자열(str) 함수

- replace() : 특정 문자(열)를 다른 문자(열)로 치환
- strip() : 문자열의 시작과 끝에 있는 공백과 줄 바꿈 기호 제거

```
# 문자열 생성
string = "Life is too short"
print(f'replace 함수 사용 전 문자열 출력 : ₩n{string}')

print('-'*80)

# replace() 적용
result = string.replace("Life", "My leg")

# 결과 확인
print(f'replace 함수 사용 후 문자열 출력 : ₩n{result}')
```

```
replace 함수 사용 전 문자열 출력 :
Life is too short
--------------------------------------------------------------------------------
replace 함수 사용 후 문자열 출력 :
My leg is too short
```

```
# 문자열 생성
string = "₩n Hello world, Python! ₩n"
print(f'strip 함수 사용 전 문자열 출력 : ₩n{string}')

print('-'*80)

# strip() 적용
result = string.strip()

# 결과 확인
print(f'strip 함수 사용 후 문자열 출력 : ₩n{result}')
```

```
strip 함수 사용 전 문자열 출력 :

 Hello world, Python!

--------------------------------------------------------------------------------
strip 함수 사용 후 문자열 출력 :
Hello world, Python!
```

반복문

- 정의: 특정 함수를 반복하여 실행하도록 제어하는 명령문
- 종류: for / while



반복문이 필요한 이유

1부터 1000까지 출력하시오.

```
print(1)
print(2)
print(3)
print(4)
print(5)
print(6)
...
```

```
1
2
3
4
5
6
```



for 반복문

- 정의: 특정 함수를 일정 횟수만큼 반복하여 실행하도록 제어하는 명령문
- 특징

1) 반복해서 데이터를 추출할 수 있는 자료 구조(리스트, 튜플, 문자열, 딕셔너리)에서 각 요소를 하나씩 가져온 후 코드 블록 실행

2) 기본 구조

```
for 변수 in 문자열(or 리스트):
    print(변수)
```

들여쓰기 (Tab, Space*4)

(colon, 콜론)

for문 실습(1)

리스트와 for 문

- 리스트의 모든 원소 출력하기

```
list_food = ["햄버거","치킨","피자"]
for food in list_food:
    print(food)
```

```
햄버거
치킨
피자
```

for문 실습(2)

문자열과 for 문

- 문자열의 모든 문자 출력하기

```
hi = "안녕하세요"
for s in hi:
    print(s)
```

```
안
녕
하
세
요
```

for문 실습(3)

range(start, stop) 함수와 for 문

- 범위(start ~ stop-1) 사이의 정수의 개수만큼 함수 실행하기

while 반복문

- 정의: 특정 조건이 참(True)인 동안 코드 블록을 반복적으로 실행하는 명령문
- 특징
  1) 반복 횟수가 명확하지 않을 때 주로 사용
  2) 조건식(Condition Expression)이 참(True)인 동안 반복
  3) 조건식을 평가한 후 코드 블록 실행 (선(先) 조건 평가, 후(後) 코드 실행)

```
while 조건식:
    반복할 코드
```

```python
# 초기화: 반복을 위한 시작 값 설정
count = 1

# 조건: count가 5보다 작거나 같은 동안 반복
while count <= 5:
    print(count) # 현재 count 값 출력
    count = count + 1 # count를 1 증가 (조건이 거짓이 되도록 변화를 줌)
```

while 반복문

- 특징

4) 종료 조건 설정: 반복을 제어하는 break 키워드(예약어) 사용

① 반복을 제어하는 break 키워드(예약어) 사용

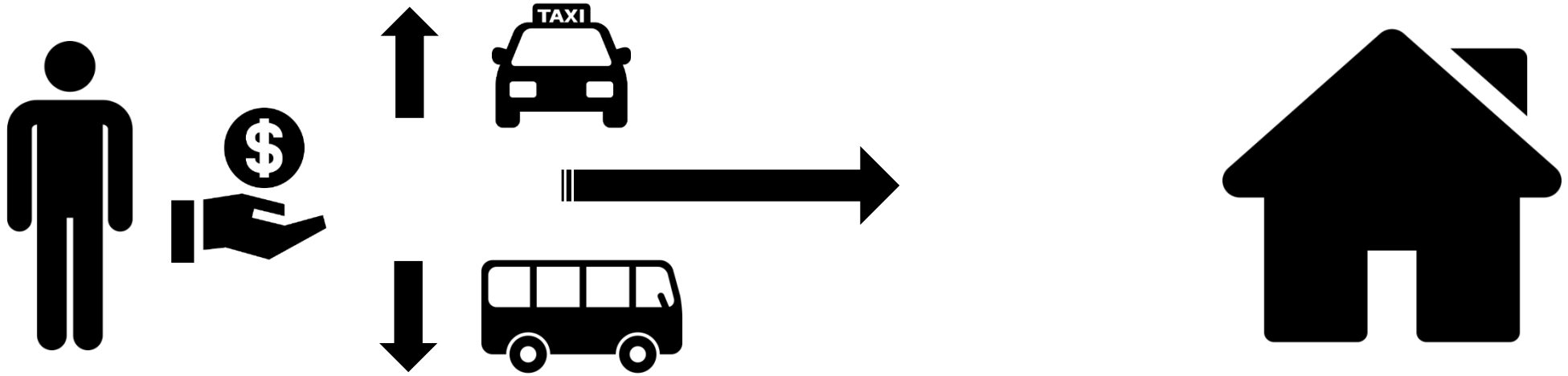
② break: 현재 실행 중인 while 반복문 즉시 종료

```
while True:
    user_input = input("사용자: ")
    if user_input.lower() in ["exit", "quit"]:
        break
    else:
        print("사용자 입력:", user_input)
```

조건문

조건문

- 상황에 따라 실행 흐름을 다르게 하는 문법



조건문 키워드

if

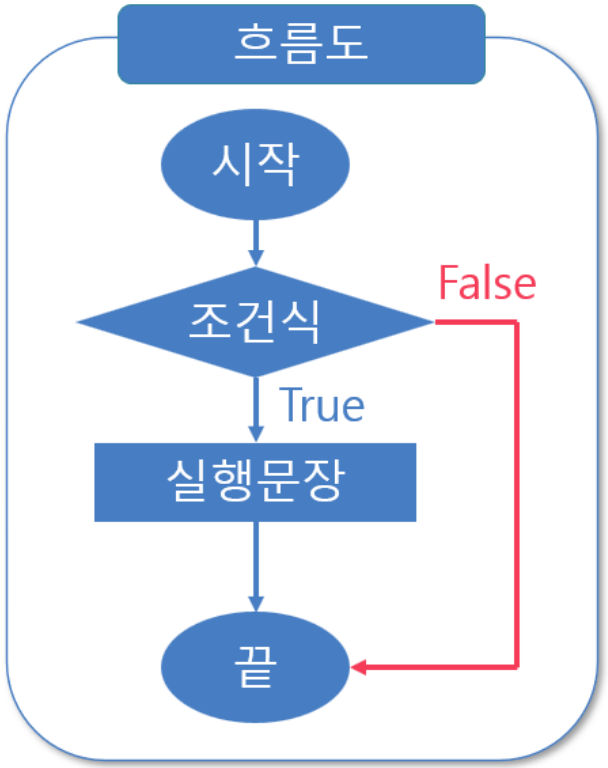
elif

else

조건문 if

if 조건식이 True일 경우 실행문장 실행

```
if 조건식:
    실행문장
    실행문장
```



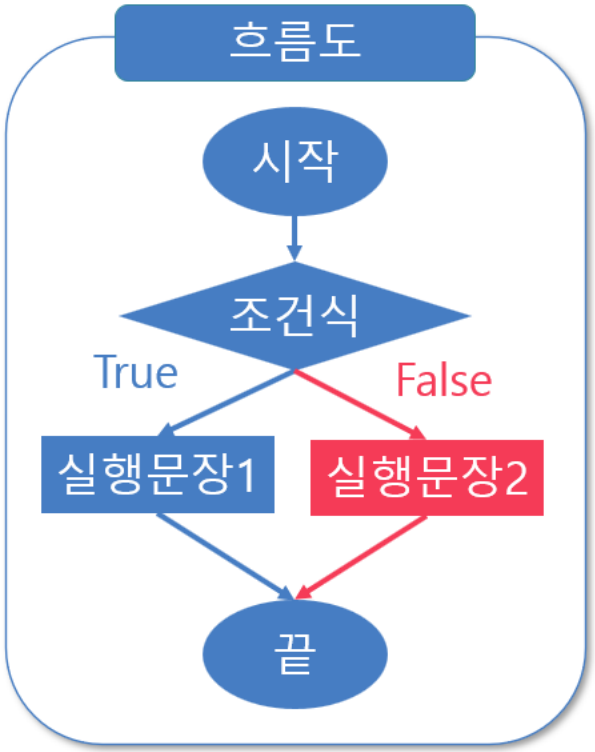
조건문 if 실습

조건문 if else

if 조건식이 True일 경우 실행문장1 실행

else 조건식이 False일 경우 실행문장2 실행

```
if 조건식:
    실행문장1
else:
    실행문장2
```



조건문 if else 실습

조건문 if elif else

- **if** 첫번째 조건식이 True일 경우 실행문장1 실행
  첫번째 조건식이 False일 경우 다음 조건식 확인
- **elif** 두번째 조건식이 True일 경우 실행문장2 실행
- **else** 두번째 조건식이 False 일 경우 실행문장3 실행

```
if 조건식1:
    실행문장1
elif 조건식2:
    실행문장2
```

```
if 조건식1:
    실행문장1
elif 조건식2:
    실행문장2
else:
    실행문장3
```

조건문 if elif else 실습

예외 처리

- 예외(exception)
  1) 코드(프로그램)를 실행하는 중에 발생하는 에러
  2) 예외가 발생 시 프로그램은 비정상 종료
     - 예시 : 10을 어떤 값으로 나누는 함수가 있을 때 인수에 따라 정상으로 동작하기도 하고 에러가 나기도 함

```
def div_ten(x):
    return 10 / x

result = div_ten(2)
print(result)
```

```
5.0
```

```
result = div_ten(0)
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback
<ipython-input-6-29a8c1c90343> in <cell line: 1>()
----> 1 result = div_ten(0)

<ipython-input-5-bed92e778184> in div_ten(x)
      1 def div_ten(x):
----> 2     return 10 / x
      3
      4 result = div_ten(2)
      5 print(result)

ZeroDivisionError: division by zero
```

예외 처리

- 예외 처리

  1) 예외가 발생했을 때 프로그램이 중단되지 않고 계속 실행되도록 코드를 작성하는 것

  2) 기본 문법 : try-except문

```
try:
    실행할 코드
except:
    예외가 발생했을 때 처리하는 코드
```

```
### 예외 처리를 안하는 경우
for i in range(10):
    print(10/i)
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
<ipython-input-14-7386f747dd3d> in <cell line: 2>()
      1 ### 예외 처리를 안하는 경우
      2 for i in range(10):
----> 3     print(10/i)

ZeroDivisionError: division by zero
```

```
# 예외 처리를 한 경우
for i in range(10):
    try:
        print(10/i)
    except:
        print("Not divived by 0")
```

```
Not divived by 0
10.0
5.0
3.3333333333333335
2.5
2.0
```

예외 처리

- 명시적 예외 처리

  : except에 예외의 이름을 지정해서 해당 예외가 발생했을 때만 프로그램이 중단되지 않도록 처리

```
try:
    실행할 코드
except 예외이름:
    예외가 발생했을 때 처리하는 코드
```

[예외의 종류]

예외	내용
IndexError	리스트의 인덱스 범위를 넘어갈 때
NameError	존재하지 않는 변수를 호출할 때
ZeroDivisionError	0으로 숫자를 나눌 때
ValueError	변환할 수 없는 문자나 숫자를 변환할 때
FileNotFoundError	존재하지 않는 파일을 호출할 때

```
for i in range(10):
    try:
        print(10/i)
    except ZeroDivisionError:
        print("Not divived by 0")
```

```
Not divived by 0
10.0
5.0
```

예외 처리

- 포괄적 예외 처리

  : 발생할 수 있는 모든 오류를 한번에 처리하는 방식

```
try:
    실행할 코드
except Exception as e:
    print("프로그램 실행 중 예측하지 못한 오류가 발생했습니다.")
    print(f"오류 유형: {type(e).__name__}")
```

파일 처리: 파일 읽고 쓰기

- 기본 문법

```
with open("파일명", "파일 열기 모드") as f:
    # 파일 읽기 / 파일 쓰기 함수
```

[파일 열기 모드]

종류	설명
r	읽기 모드: 파일을 읽기만 할 때 사용
w	쓰기 모드: 파일에 내용을 쓸 때 사용
a	추가 모드: 파일의 마지막에 새로운 내용을 추가할 때 사용

파일 처리: 파일 읽고 쓰기

- 파일 읽기

1) 파일 열기 모드 'r'로 변수 생성

2) 변수.read() 함수 사용 : 파일 내용 전체를 문자열 자료형으로 전달

3) 변수.readlines() 함수 사용 : 파일에 있는 각각의 줄을 요소로 가지는 리스트 자료형 전달

```
# 파일 읽기: with-open 구문 + 'r' mode

## 파일 경로 설정
file_path = 'dream.txt'

## with-open 구문 + read()
with open(file=file_path, mode='r') as f:
    contents = f.read()
    print(contents)
```

파일 처리: 파일 읽고 쓰기

- 파일 쓰기

1) 파일 열기 모드 'w'로 파일 변수 생성

→ 중복된 파일이 있을 경우 기존 파일을 무시하고 내용을 새로 입력

2) 변수.write() 함수 사용 : 새로운 파일에 내용 입력

```
# 파일 쓰기: with-open 구문 + 'w' mode

## 저장할 파일 경로 설정
file_path='count_log.txt'

## 파일 쓰기
with open(file=file_path, mode='w', encoding='utf8') as f:
    for num in range(1,11):
        data = f"{num}번째 줄입니다.₩n"
        f.write(data)
```

클래스(class): 세상에 없던 '나만의 자료형(커스텀 자료형)'만들기

- 우리가 이미 써온 자료형들

  : 파이썬이 기본 제공하는 내장 자료형(Built-in Type): int, float, str, bool, list, tuple, dict

- 클래스 (커스텀 자료형)

  : 기본 자료형만으로는 표현하기 어려운 현실 세계의 데이터를 표현하기 위해 기존에 없던 새로운 종류의 데이터 타입(Custom Data Type)을 직접 정의하는 것

클래스와 데이터의 관계: '붕어빵'틀과 '붕어빵'



```
# 리스트 클래스 -> 리스트 데이터 생성
list_data1 = []
list_data2 = [10, 20]

# 딕셔너리 클래스 -> 딕셔너리 데이터 생성
dict_data1 = {}
dict_data2 = {'name': 'MH'}
```

- 붕어빵 틀(클래스)은 하나지만, 거기서 찍어내어 변수에 담는 붕어빵(데이터)들은 저마다 다른 값을 가질 수 있음

[구체적 예시] 고객(Customer) 자료형 정의

```
# 1. 'Customer(고객) ' 이라는 이름의 커스텀 자료형 '틀(설계도)' 만들기
class Customer:
    # 이 자료형이 만들어질 때 가질 세부 데이터(속성)를 정한다
    def __init__(self, name, age, gender):
        self.name = name        # 고객 이름 데이터
        self.age = age          # 고객 나이 데이터
        self.gender = gender    # 고객 성별 데이터

    # 이 자료형만 사용할 수 있는 전용 기능(메서드)을 정한다
    def purchase(self, price, product, name):
        print(f"[{self.name}] 고객님이 {price}원 상당의 {product_name}를 구매하셨습니다.")

# 2. 'Customer’ 클래스 이용해 구체적인 데이터 생성하기
    client = Customer("김철수", 24, "남성")
```

변수에서 데이터 꺼내 쓰고 기능(메서드) 실행하기

- 변수 이름 끝에 점(.)을 찍고 저장된 데이터를 꺼내거나 전용 기능 실행
  1) 데이터 꺼내기

```
print(customer1.name)   # 결과: 김철수
print(customer2.age)    # 결과: 27
```

  2) 전용 기능 실행하기

```
client.purchase(900000, "아이패드" )
# 결과: 김철수 고객님이 900000원 상당의 아이패드를 구매하셨습니다
```

- 변수의 '자료형' 확인하기 (type() 함수)

```
print(type(client))    # 결과: <class '__main__.Customer'>
```

메서드(클래스 전용 함수) 사용법

1. 클래스 호출 → 데이터 생성

```
client = Customer(name, gender, age)
```

2. 메소드 호출

```
output = client.purchase(price, product_name)
```

클래스 상속(Inheritance)

- 정의

  : 기존 클래스의 속성과 메서드를 그대로 물려받아, 새로운 속성을 추가하거나 기능을 수정하여 클래스를 확장하는 기술

- 부모 클래스와 자식 클래스 (누가 부모이고, 누가 자식인가?)

  1) 부모 클래스 (Parent / Super Class)

  ① 역할: 공통된 속성과 기능을 정의하여 자식에게 '물려주는' 원본 설계도이다.

  ② 예시: Customer 클래스 (모든 고객이 공통으로 가지는 이름, 나이, 구매 기능 보유)

  2) 자식 클래스 (Child / Sub Class)

  ① 역할: 부모의 기능을 '물려받아' 필요한 속성을 추가하거나 기능을 확장하는 클래스입니다.

  ② 예시: VIPCustomer 클래스 (Customer의 기능을 모두 가지면서 '할인율' 속성과 'VIP 구매' 기능을 추가)'

클래스 상속(Inheritance)

- 상속을 사용하는 이유 (왜 필요한가?)
  1) 코드 재사용성(Reusability): 중복 코드를 새로 작성할 필요 없이 부모의 코드를 재활용한다.
  2) 유지보수성(Maintainability): 공통 로직이 변경될 경우 부모 클래스 한 곳만 수정하면 된다.
  3) 확장성(Extensibility): 기존 부모 코드를 건드리지 않고 자식 클래스에서 안전하게 새로운 기능을 추가할 수 있다.

[구체적 예시] 일반 고객(Customer)에서 VIP 고객(VIPCustomer)으로 확장

- 상속 기본 문법

```python
# 1. 부모 클래스: 기존 Customer 클래스 정의
class Customer:
    def __init__(self, name, age, gender):
        self.name = name        # 고객 이름
        self.age = age          # 고객 나이
        self.gender = gender    # 고객 성별

    def purchase(self, price, product_name):
        print(f"[{self.name}] 고객님이 {price:,}원 상당의 {product_name}를 구매하셨습니다.")

# 2. 자식 클래스: Customer(부모)를 상속받는 VIPCustomer(자식) 클래스 정의
class VIPCustomer(Customer):
    def __init__(self, name, age, gender, discount_rate=0.1):
        # super(): 부모 클래스의 __init__() 생성자를 호출하여 기본 속성 초기화
        super().__init__(name, age, gender)
        self.discount_rate = discount_rate  # VIP 전용 추가 속성 (할인율)
```

메서드 오버라이딩 (Method Overriding)

- 개념; 부모 클래스로부터 물려받은 메서드를 자식 클래스의 로직에 맞게 덮어쓰기(재정의)하는 것이다.

```
class VIPCustomer(Customer):
    def __init__(self, name, age, gender, discount_rate=0.1):
        super().__init__(name, age, gender)
        self.discount_rate = discount_rate

    # [메서드 오버라이딩] 부모의 purchase() 메서드를 VIP용 구매 로직으로 재정의
    def purchase(self, price, product_name):
        # 할인된 가격 계산
        discount_price = int(price * (1 - self.discount_rate))
        print(f"""[VIP] [{self.name}] 고객님이 {self.discount_rate * 100:.0f}% 할인받아
        {discount_price:,}원에 {product_name}를 구매하셨습니다.""")
```

실습: 상속된 기능 실행하기

라이브러리 (Library), 패키지 (Package), 모듈 (Module)

- 개념: Python에서 코드 재사용 및 효율적인 관리를 위한 단위
- 주요 특징 및 관계

1) 모듈 (Module)

① 가장 작은 코드 재사용 단위

② 하나의 .py 파일이 하나의 모듈

③ 변수, 함수, 클래스 등을 포함

④ 점(.)을 사용하여 모듈 내부의 함수나 변수에 접근할 수 있음 예) 모듈.함수() / 모듈.변수

2) 패키지 (Package)

① 관련된 여러 모듈들을 묶어 놓은 디렉토리(폴더)

② 점(.)을 사용하여 하위 모듈이나 하위 패키지에 접근할 수 있음 예) 패키지.모듈, urlib.request

라이브러리 (Library), 패키지 (Package), 모듈 (Module)

- 정의: Python에서 코드 재사용 및 효율적인 관리를 위한 단위
- 주요 특징 및 관계

3) 라이브러리 (Library)

① 모듈과 패키지들을 체계적으로 모아 놓은 집합을 포괄적으로 일컫는 용어

② '라이브러리'라는 명확한 문법적 단위가 있는 것은 아니며, 일반적으로 패키지와 혼용해서 사용

③ 파이썬 표준 라이브러리: 파이썬 설치 시 기본적으로 제공되는 방대한 모듈과 패키지들의 집합

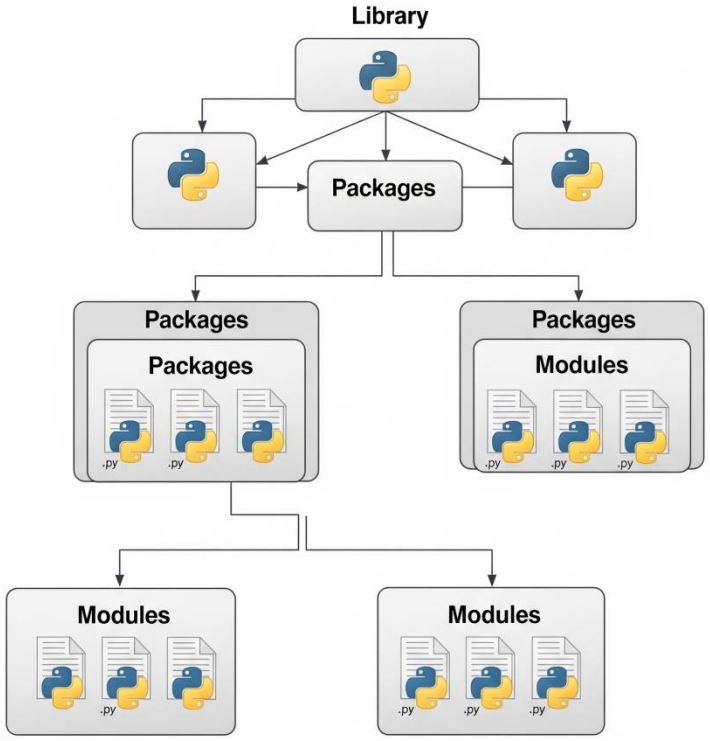
④ 외부 라이브러리

- 파이썬 패키지 관리 프로그램인 pip를 사용하여 설치하여 사용하는 라이브러리
- 대표적인 외부 라이브러리
  - 시각화 라이브러리 : matplotlib, seaborn, plotly
  - 데이터 분석 : pandas
  - 복잡한 수학 연산 처리 : numpy
  - 머신 러닝 : scikit-learn, autogluon, pycaret
  - 딥러닝 : tensorflow, pytorch, transformers, sentence-transformers

라이브러리 (Library), 패키지 (Package), 모듈 (Module)

- 정의: Python에서 코드 재사용 및 효율적인 관리를 위한 단위
- 주요 특징 및 관계

  4) 관계 요약: 모듈 < 패키지 < 라이브러리 순으로 포함 관계가 확장)



라이브러리 (Library), 패키지 (Package), 모듈 (Module)

- 모듈(module)에 정의된 함수 사용법

1) python 내장 모듈 : 파이썬 표준 라이브러리에 내장되어 있는 모듈

① 별도의 설치는 필요 없지만 사용할 때,

- 모듈 import 후 함수 호출
- 패키지.모듈 import 후 함수 호출

② 예시

```
### python 내장 모듈 - random 모듈을 이용한 로또 번호 추출

# 필요한 모듈 임폴트
import random

# 1부터 45까지의 정수 범위 설정 (45를 포함하기 위해 46까지 지정)
numbers = range(1, 46)

# 범위에서 중복 없이 6개의 숫자를 선택
lottery = random.sample(numbers, 6)

# 선택된 숫자 출력
print(lottery)
```

라이브러리 (Library), 패키지 (Package), 모듈 (Module)

- 모듈(module)에 정의된 함수 사용법

  2) 외부 모듈 : 외부 라이브러리에 들어있는 파이썬 파일

  ① 해당 라이브러리(패키지)를 설치 → pip install library(package)

  - 라이브러리 import
  - 함수 호출

  ② 예시

```
### 필요한 라이브러리 임폴트
import pandas as pd

### read_csv() 호출
df = pd.read_csv(file_path)
```

03. streamlit

streamlit 개요

- Streamlit이란?

  : 복잡한 웹 개발 언어(HTML, CSS, JavaScript)나 백엔드 서버 지식 없이, 순수 파이썬 만으로 웹 사이트를 만들어주는 프로그램

- 왜 사용하는가?

  : 웹 화면 하나를 만들려면 별도의 웹 개발 공부가 필요하다. 하지만 Streamlit을 사용하면 파이썬 변수와 함수 몇 줄로 단 몇 분 만에 완성도 높은 대시보드 웹을 완성할 수 있다.

- 사용 방법

  1) 설치: pip install streamlit (터미널에 입력)
  2) 임포트: import streamlit as st (파이썬 코드 상단에 작성)
  3) 함수 호출: st.func()
  4) 실행: streamlit run 파일명.py (터미널에 입력하여 웹 브라우저 실행)

streamlit 기능: 텍스트 출력

- 웹 화면에 제목, 본문, 안내 문구 등의 글자를 출력해 주는 기능
- 주요 텍스트 기능 구현 함수
  1) st.title(): 가장 큰 메인 제목을 만들 때 사용
  2) st.header(): 대분류 주제 제목을 만들 때 사용
  3) st.subheader(): 중분류 세부 제목을 만들 때 사용
  4) st.write(): 파이썬의 print()처럼 무엇이든(문자, 숫자, 딕셔너리 등) 알아서 출력해 주는 만능 함수
  5) st.caption(): 화면 하단의 작은 주석이나 부연 설명을 쓸 때 사용

streamlit 기능: 텍스트 출력 실습

streamlit 기능: 사용자 입력 위젯 생성

- 위젯(Widget)
  1) 사용자가 마우스로 클릭하고, 드래그하고, 글자를 타이핑하는 등 화면과 상호작용(Interaction)하며 데이터를 조작하거나 탐색할 수 있도록 돕는 UI(사용자 인터페이스) 요소
  2) 사용자 입력 위젯: 일방적으로 정보만 보여주는 정적인 웹페이지가 아니라, 사용자가 누른 값에 따라 다르게 반응하는 '동적인 웹 서비스'를 만들기 위해 사용한다.

streamlit 기능: 사용자 입력 위젯 생성

- 주요 입력 위젯 기능 구현 함수

1) 선택형 입력 기능 구현 함수:

① st.button("버튼명"): 클릭하면 True, 안 누르면 False를 반환하는 버튼

② st.checkbox("선택문구"): 체크 여부(True/False)를 확인하는 체크박스

③ st.radio("제목", 리스트): 동그라미 버튼 중 딱 하나만 고르게 하는 함수

④ st.selectbox("제목", 리스트): 드롭다운 메뉴로 하나만 고르게 하는 함수

⑤ st.multiselect("제목", 리스트): 여러 개를 동시에 선택하게 하는 함수 (결과: 파이썬 list 반환)

2) 값 입력 기능 구현 함수

① st.text_input("제목"): 사용자가 글자(문자열)를 직접 입력하는 창

② st.number_input("제목"): 사용자가 숫자를 입력하는 창 (정수/실수 반환)

streamlit 기능: 사용자 입력 위젯 생성 실습

streamlit 기능: 데이터 표출 & 차트 생성

- 개념

: 파이썬 딕셔너리(dict)나 리스트(list) 데이터를 웹 화면에 숫자 카드, 정돈된 표, 시각적 그래프로 변환하여 출력해 주는 기능

- 목적

: 복잡한 숫자 데이터를 시각적인 표와 그래프로 보여주어 데이터의 의미를 직관적으로 전달하기 위해 사용

- 주요 데이터 표출 및 차트 기능 구현 함수

1) 주요 수치 카드 (st.metric):

: 매출액, 회원 수 같은 핵심 성과 지표(KPI)와 이전 대비 변화량(Delta)을 하이라이트하여 크게 보여준다.

2) 파이썬 데이터 표 출력 (st.table):

: 파이썬의 [{"이름": "김철수", "나이": 25}, ...] 구조를 깔끔한 웹 표 형태(Table)로 직접 그려준다.

3) 내장 차트 출력 (st.bar_chart, st.line_chart)

: Pandas 같은 외부 분석 도구 없이도 파이썬 딕셔너리 {"1월": 100, "2월": 200} 형태를 전달하면 즉시 막대그래프나 선그래프로 그려준다.

streamlit 기능: 데이터 표출 & 차트 생성 실습

streamlit 기능: 레이아웃 & 컨테이너 생성

- 개념

: 웹 화면을 좌우로 나누거나(컬럼), 왼쪽 구석에 메뉴판을 만들거나(사이드바), 탭으로 분할해 주는 '화면 정리 정돈' 기능

- 목적

: 입력창과 글자가 위에서 아래로만 길게 배치되면 화면이 지저분해짐. 화면 공간을 효율적으로 분할하여 사용자가 이용하기 편한 깔끔한 UI를 제공하기 위해 사용

- 주요 레이아웃 기능 구현 함수

1) 사이드바 (st.sidebar): 화면 왼쪽에 메뉴나 설정창을 숨겨두는 공간을 만든다.
2) 다단 컬럼 (st.columns): 화면을 가로로 2개, 3개 등 원하는 비율로 나란히 분할한다.
3) 탭 나누기 (st.tabs): 한 화면 안에서 탭(Tab)을 눌러 서로 다른 내용을 전환하며 보여준다.
4) 접기/펼치기 (st.expander): 긴 설명글을 접어두었다가 사용자가 클릭할 때만 펼쳐지게 만든다.

streamlit 기능: 레이아웃 & 컨테이너 생성 실습

streamlit 기능: 세션 상태(st.session_state)를 이용한 메모리

- 개념

: 웹 브라우저가 열려있는 동안 파이썬 변수의 값을 기억하고 보존해 두는 메모리 역할의 도구이다.

- 목적

1) Streamlit은 사용자가 버튼 하나만 눌러도 파이썬 스크립트 전체를 처음부터 끝까지 다시 실행한다.
2) 이때 일반 파이썬 변수(예: a = 10)는 0으로 초기화되어 데이터가 소실된다.
3) 새로고침이 일어나도 내가 저장한 고객 목록이나 카운트 숫자가 날아가지 않고 유지되게 하려면 반드시 st.session_state에 저장해야 한다.

streamlit 기능: 세션 상태(st.session_state)를 이용한 메모리

- 사용법

1) st.session_state는 파이썬 딕셔너리(dict)와 사용법이 동일하다.

2) 존재하지 않는 키를 불러오면 에러가 나므로 if 'key' not in st.session_state: 조건문으로 먼저 존재 여부를 검사하는 것이 필수 규칙이다.

```python
# 1. 세션 상태 안전 초기화 (처음 웹에 들어왔을 때 1번만 실행)
if "count" not in st.session_state:
    st.session_state["count"] = 0  # 초기값 0 세팅

# 2. 버튼을 누르면 세션 상태 내부의 값을 1 증가
if st.button("숫자 1 증가"):
    st.session_state["count"] = st.session_state["count"] + 1

# 3. 화면이 재실행되어도 변하지 않고 유지되는 값 출력
st.write(f"현재 누적 카운트: **{st.session_state['count']}**")
```

streamlit 기능: 세션 상태(st.session_state)를 이용한 메모리 실습

streamlit 기능: 데이터 CRUD

- 개념

1) Streamlit은 데이터가 변하는 흐름을 웹 화면에 실시간으로 일치(동기화)시켜 주는 도구이다.

2) 개발자가 복잡한 웹 화면 구현 코드를 짜지 않아도, Streamlit은 데이터의 상태에 따라 다음과 같이 화면을 알아서 바꾸어 준다.

① Create (생성) & Read (조회): 사용자가 입력창에 데이터를 입력하여 추가하면, Streamlit은 이를 즉시 감지하여 화면에 새로운 데이터를 바로 띄워준다.

② Update (수정): 사용자가 기존의 데이터를 수정하는 순간, Streamlit은 수정된 데이터를 반영하여 화면의 내용을 자동으로 변경해 준다.

③ Delete (삭제): 사용자가 데이터를 삭제하면, Streamlit은 해당 데이터가 화면에서 깔끔하게 사라지도록 실시간으로 화면을 업데이트한다.

streamlit 기능: 데이터 CRUD

- 파이썬 기본 자료형(list, dict)을 활용한 CRUD 구현

1) Create (추가): 입력 위젯에서 받은 값을 dict로 묶어서 세션 리스트에 .append() 한다.

2) Read (조회): 세션 리스트 데이터를 출력한다.

```python
# 세션 상태에 고객 리스트 초기화
if "customer_list" not in st.session_state:
    st.session_state["customer_list"] = []

# C (Create): 데이터 추가
new_name = st.text_input("고객 이름 입력")
if st.button("고객 추가"):
    st.session_state["customer_list"].append({"name": new_name})
    st.success(f"{new_name} 고객이 등록되었습니다.")

# R (Read): 데이터 조회
st.write("현재 등록된 고객 목록:", st.session_state["customer_list"])
```

streamlit 기능: 데이터 CRUD 실습

streamlit 기능: 버벅임 없는 데이터 전송, 입력 폼(st.form)

- 개념

  1) st.form 함수: 여러 개의 입력 위젯을 하나로 묶어, 오직 [제출 버튼]을 누르는 순간에만 데이터가 전송되고 화면이 새로고침되도록 제어하는 '입력 폼(제출 양식)' 기능 구현 함수이다.

  2) st.form_submit_button: st.form 함수: 내부의 모든 입력 데이터를 한 번에 서버로 전송하고 화면을 새로고침(Rerun)시키는 전용 제출 버튼 기능 구현 함수이다.

- 목적

  : 입력창 하나 칠 때마다 웹 화면이 새로고침되는 것을 방지한다. 입력창이 5개라면 화면이 5번 새로고침되면서 데이터 연산이나 함수 호출이 반복됩니다. st.form은 이를 차단하여 앱이 버벅거리는 현상을 막아준다.

streamlit 기능: 버벅임 없는 데이터 전송, 입력 폼(st.form) 실습

streamlit 기능: 환경(색상, 폰트) 설정

- config.toml 설정 파일

1) 개념: 웹 앱의 글꼴(Font), 배경색, 메인 강조 색상(Primary Color) 등 전체적인 웹 디자인 테마를 한곳에서 통합 관리하는 '디자인 환경 설정 파일'이다.

2) 사용 목적: 파이썬 코드를 일일이 수정하지 않고, 단 하나의 설정 파일 수정만으로 웹 사이트 전체에 회사 브랜드 색상이나 세련된 현대적 디자인을 일괄 적용할 수 있다.

3) 파일 저장 위치 (중요 규칙)

① 반드시 실습 폴더 최상단에 .streamlit이라는 이름의 폴더를 만들고, 그 안에 config.toml 파일명으로 생성해야 Streamlit이 자동으로 디자인을 인식한다.

② 경로 구조: [내 프로젝트 폴더] > [.streamlit] > [config.toml]

```
my_dashboard/
├── .streamlit/
│   └── config.toml   <-- 테마 설정 파일
└── app.py            <-- 테마 확인용 파이썬 코드
```

streamlit 기능: 환경(색상, 폰트) 설정

- [theme]

1) 개념: 애플리케이션의 시각적 스타일을 정의하는 섹션이다.

```
# .streamlit/config.toml

[theme]
# 메인 강조 색상 (버튼, 선택 탭, 슬라이더 등)
primaryColor = "#2563EB"

# 메인 화면 배경색 (눈이 편안한 오프화이트)
backgroundColor = "#F8FAFC"
```

2) 주요 디자인 설정 항목

① primaryColor: 버튼, 선택된 탭, 슬라이더 등 가장 강조되어야 하는 포인트 색상을 지정한다.

② backgroundColor: 웹 페이지 메인 화면의 기본 배경 색상을 지정한다.

③ secondaryBackgroundColor: 사이드바(Sidebar), 입력 상자 영역 등의 보조 배경 색상을 지정합니다.

④ textColor: 화면에 표시되는 기본 글자(텍스트) 색상을 지정합니다.

⑤ font: 기본 글꼴 스타일을 지정합니다 (sans serif, serif, monospace 중 선택)

streamlit 기능: 환경(색상, 폰트) 설정

- 실무 권장: 2가지 타입의 모던 디자인 테마 추천

1) 타입 A: 모던 화이트 (Modern Light Pro) — [기업용 깔끔한 스타일]

① 눈의 피로도를 줄여주는 오프화이트 배경과 신뢰감을 주는 모던 블루 강조 색상의 조합이다.

② primaryColor = "#2563EB" (인디고 블루)

③ backgroundColor = "#F8FAFC" (눈이 편안한 소프트 화이트)

④ secondaryBackgroundColor = "#EFF6FF" (연한 블루 톤 사이드바)

⑤ textColor = "#1E293B" (시독성 높은 딥 차콜)

2) 타입 B: 프리미엄 다크 (Premium Dark) — [데이터 분석가 선호 스타일]

① 데이터 시각화 그래프가 가장 돋보이는 모던 다크 모드 스타일이다.

② primaryColor = "#3B82F6" (밝은 블루)

③ backgroundColor = "#0F172A" (딥 다크 블루)

④ secondaryBackgroundColor = "#1E293B" (차콜 다크 사이드바)

⑤ textColor = "#F8FAFC" (화이트 글자)

streamlit 기능: 환경(색상, 폰트) 설정 실습


MISSION:
COMPLETE